

UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Programmazione 2

Primo parziale

A cura di:

Mirco Negri

Aprile 2026

Indice

1	Lezione 01: Recap delle nozioni di Programmazione 1	5
1.1	Spiegazione teorica	5
1.2	Implementazione del codice	6
1.2.1	Test sulle Variabili (<code>test_vars.cpp</code>)	6
1.2.2	Test sullo Stack Frame (<code>test_stack_frame.cpp</code>)	6
1.2.3	Test sui Puntatori e Modifiche (<code>test_puntatori.cpp</code>)	6
1.2.4	Test sull'Allocazione Dinamica (<code>test_memoria.cpp</code>)	7
1.3	Esercizi e approfondimenti dal tutorato	7
2	Lezione 02: Introduzione all'Orientamento agli Oggetti	7
2.1	Spiegazione teorica	7
2.2	Implementazione del codice	8
2.2.1	Versione 1: Struttura base e Costruttore	8
2.2.2	Versione 2 e 3: Aggiunta di comportamento (Metodi)	8
2.2.3	Uso delle Enum (<code>Direction.java</code>)	9
2.3	Esercizi e approfondimenti dal tutorato	9
2.3.1	Modellazione gerarchica di oggetti (<code>Settings</code>)	9
3	Lezione 03: Organizzazione del codice e pattern base	10
3.1	Spiegazione teorica	10
3.2	Implementazione del codice	12
3.2.1	Immutabilità con <code>final</code> (<code>ToolTier.java</code>)	12
3.2.2	Variabili di gioco globali e <code>static</code> (<code>GameConstants.java</code>)	13
3.2.3	Il Singleton Pattern (<code>DragonEgg.java</code>)	13
3.3	Esercizi e approfondimenti dal tutorato	14
4	Lezione 04: Incapsulamento, Invarianti e Memory Layout	14
4.1	Implementazione del codice	15
4.1.1	Protezione degli Invarianti	15
4.1.2	Passaggio dei valori per riferimento e per copia	15
4.1.3	Layout degli oggetti e memoria	16
4.2	Esercizi e approfondimenti dal tutorato	16
5	Lezione 05: Ereditarietà e Polimorfismo di Sottotipo	16
5.1	Spiegazione teorica	16
5.2	Implementazione del codice	17
5.2.1	La Superclasse (<code>Entity.java</code>)	17
5.2.2	La Sottoclasse (<code>LivingEntity.java</code>)	18
5.3	Esercizi e approfondimenti dal tutorato	18
5.3.1	Gerarchia degli Oggetti (<code>es5</code>)	18
6	Lezione 06: Astrazione e Polimorfismo Ad-hoc	19
6.1	Spiegazione teorica	19
6.2	Implementazione del codice	20
6.2.1	Classi e Metodi Astratti (<code>AbstractEntity.java</code>)	20
6.2.2	Overloading vs Overriding (<code>OverloadTest.java</code>)	20
6.3	Esercizi e approfondimenti dal tutorato	21

6.3.1	Esercizio: La gerarchia Flatland	21
7	Lezione 07: Interfacce e Capacità	21
7.1	Spiegazione teorica	21
7.2	Implementazione del codice	22
7.2.1	Definizione di un'interfaccia (<code>Cancellable.java</code>)	22
7.2.2	Implementazione multipla	22
7.3	Esercizi e approfondimenti dal tutorato	23
7.3.1	Sistemi di pagamento polimorfici	23
8	Lezione 08: Composizione vs Ereditarietà e lo Strategy Pattern	23
8.1	Spiegazione teorica	23
8.2	Implementazione del codice	24
8.2.1	L'Interfaccia Strategy (<code>AttackStrategy.java</code>)	24
8.2.2	Le Strategie Concrete	25
8.2.3	La Classe di Contesto (<code>Piglin.java</code>)	26
8.2.4	Modifica del comportamento a Runtime (<code>Lecture8.main</code>)	27
9	Lezione 09 (Lab 1): Modellazione, Identità degli oggetti e gerarchie	27
9.1	Spiegazione teorica e Struttura del Laboratorio	27
9.2	Implementazione pratica: Esercizio di Modellazione	28
9.3	Esercizi e approfondimenti dal tutorato	28
10	Lezione 10: Il Dispatch (Static vs Dynamic Binding) e la risoluzione degli argomenti	29
10.1	Spiegazione teorica	29
10.2	Implementazione del codice	30
10.2.1	Static vs Dynamic Binding (<code>Block</code> e <code>DiamondBlock</code>)	30
10.2.2	La risoluzione degli argomenti (<code>Miner</code> e <code>Pick</code>)	31
10.3	Esercizi e approfondimenti dal tutorato	31
11	Lezione 11: V-Table, Casting, instanceof e Template Method	32
11.1	Spiegazione teorica	32
11.2	Implementazione del codice	33
11.2.1	Il problema: L'abuso di <code>instanceof</code> (<code>BadBlock.java</code>)	33
11.2.2	La soluzione: Il Template Method (<code>AbstractBlock.java</code>)	34
11.2.3	Le implementazioni concrete	34
11.3	Esercizi e approfondimenti a lezione	35
12	Lezione 12 (Lab 2): Classi astratte, Metodi e Refactoring	35
12.1	Spiegazione teorica e Struttura del Laboratorio	35
12.2	Implementazione pratica: Esercizio di Refactoring	36
12.3	Esercizi e approfondimenti dal tutorato	37
13	Lezione 13: Gestione degli Errori ed Eccezioni	37
13.1	Spiegazione teorica	37
13.2	Implementazione del codice	38
13.2.1	Definizione di una gerarchia di Eccezioni	38
13.2.2	Sollevare e dichiarare le eccezioni (<code>CraftingTable.java</code>)	39

13.2.3	Catturare e gestire gli errori	39
13.2.4	Eccezioni e Costruttori	40
13.3	Esercizi e approfondimenti dai laboratori (Lab 3)	40
14	Lezione 14: Modellazione e Spawner	40
14.1	Spiegazione teorica	40
14.2	Implementazione del codice	41
14.2.1	La classe Spawner (<code>Spawner.java</code>)	41
14.3	Esercizi e approfondimenti dal tutorato	42
14.3.1	Modellazione di Sistemi Gerarchici (Sonda)	42
15	Lezione 15: Creazione ed Evoluzione (Factory Method e State Pattern)	42
15.1	Spiegazione teorica	42
15.2	Implementazione del codice	43
15.2.1	Il Factory Method (<code>MobSpawner</code>)	43
15.2.2	Lo State Pattern (<code>Creeper</code>)	44
15.3	Esercizi e approfondimenti dal tutorato	45
16	Lezione 16 (Lab 3): Esercitazione su eccezioni, generics e pattern	45
16.1	Spiegazione teorica e Struttura del Laboratorio	45
16.2	Implementazione pratica: Classe Generica Sicura	46
16.3	Esercizi e approfondimenti dal tutorato	47
16.3.1	Il Sistema QuestLog e le Eccezioni di Dominio	47
17	Lezione 17: Uguaglianza e Ordinamento	47
17.1	Spiegazione teorica	47
17.2	Implementazione del codice	48
17.2.1	Override sicuro di <code>equals</code> e <code>hashCode</code>	48
17.2.2	Implementazione di <code>Comparable</code> (Ordinamento Naturale)	49
17.2.3	Utilizzo di <code>Comparator</code> (Ordinamento Alternativo)	49
17.3	Esercizi e approfondimenti dai laboratori	50
18	Lezione 18: Comportamenti reattivi e operazioni esterne (Observer e Visitor)	50
18.1	Spiegazione teorica	50
18.2	Implementazione del codice	51
18.2.1	L'Observer Pattern (<code>Player</code> e <code>AchievementSystem</code>)	51
18.2.2	Il Visitor Pattern (<code>Entity</code> e <code>DamageVisitor</code>)	52
18.3	Collegamento con i tutorati e sviluppi futuri	52
19	Lezione 19 (Lab 4): Esercitazione su Collections e Pattern	53
19.1	Spiegazione teorica e Struttura del Laboratorio	53
19.2	Implementazione pratica: Collections e Ordinamento	53
19.3	Esercizi e approfondimenti dal tutorato (Lab 4)	55
19.3.1	Esercizio 1: Palestra e Categorie	55
19.3.2	Esercizio 3: Dispositivi e Tecnici (Verso il Visitor)	55

20 Lezione 20: Programmazione Funzionale (Interfacce Funzionali e Lambda)	56
20.1 Spiegazione teorica	56
20.2 Implementazione del codice	57
20.2.1 Refactoring: Dalle Classi Anonime alle Lambda	57
20.2.2 Uso pratico di Predicate e Consumer	58
20.3 Collegamento con i laboratori futuri (Verso le GUI)	58
21 Lezione 21: Architettura del Software (Pattern Model-View-Controller)	59
21.1 Spiegazione teorica	59
21.2 Implementazione del codice	59
21.2.1 La Vista (Intercettare l'input visivo)	60
21.2.2 Il Controller (La logica di smistamento)	61
21.3 Esercizi e approfondimenti dal tutorato	61
22 Lezione 22 (Lab 5): Esercitazione su Layout grafici e implementazione MVC	62
22.1 Spiegazione teorica e Struttura del Laboratorio	62
22.2 Esercizi e approfondimenti dal tutorato (Lab 5)	62
22.2.1 Architettura dei Package (Il "Segreto" per l'Esame)	62
23 Lezione 23 (Lab 6): Conclusione ed Esercitazione Finale	64
23.1 Spiegazione teorica e Struttura del Laboratorio	64
23.2 Caso Studio Riassuntivo: Il Sistema QuestLog (Tutorato 6)	64
23.2.1 1. Il Modello: Ereditarietà e Invarianti	65
23.2.2 2. Il Modello: Gestione delle Collezioni	66
23.2.3 3. Il Controller: Il Direttore d'Orchestra	66
23.3 Conclusione e Consigli per l'Esame	67

1 Lezione 01: Recap delle nozioni di Programmazione

1

1.1 Spiegazione teorica

Fino a questo momento, in Programmazione 1, l'approccio principale è stato il "*programming in the small*": scrittura di piccole procedure e familiarizzazione con le basi del coding. Il corso di Programmazione 2 sposta il focus sul "*programming in the large*".

Quando i progetti diventano enormi (migliaia di file e linee di codice), sorgono nuove sfide:

- **Suddivisione del lavoro:** il codice deve poter essere scritto da persone e team diversi (*divide et impera*).
- **Manutenibilità:** il codice deve essere comprensibile e modificabile anche a distanza di mesi o anni senza "rompere" funzionalità esistenti.
- **Robustezza:** prevenzione e gestione strutturata degli errori.

Per ottenere questi risultati, ci si affida a buone tecniche di programmazione, ai principi dell'Ingegneria del Software e, soprattutto, al supporto fornito dai linguaggi orientati agli oggetti (**Object-Oriented, OO**), che sono il vero fulcro di questo corso.

Per comprendere a fondo i meccanismi di Java (il linguaggio principale del corso), è essenziale costruire una **Notional Machine** corretta. Si tratta di un'astrazione mentale che ci aiuta a visualizzare cosa accade "dietro le quinte" (il runtime) quando il codice viene eseguito. A tal fine, ripassiamo quattro concetti fondamentali ereditati dal C/C++:

1. **Variabili e Scope:** Le variabili possiedono un tipo e un valore di inizializzazione. Lo *scope* ne definisce la visibilità: se dichiarata fuori dalle funzioni è *globale* (visibile ovunque), se dichiarata all'interno è *locale*. I linguaggi moderni e sicuri forzano l'inizializzazione per evitare bug subdoli.
2. **Stack Frame (Record di attivazione):** Ogni volta che si invoca una funzione, viene allocato uno "stack frame" sulla memoria Stack. Questo blocco contiene i *parametri attuali*, le *variabili locali* e l'*indirizzo di ritorno* (per sapere dove riprendere l'esecuzione al termine della funzione).
3. **Puntatori:** Un puntatore è una variabile che contiene un indirizzo di memoria. L'operatore `&` estrae l'indirizzo di una variabile, mentre l'operatore `*` (dereferenziazione) permette di accedere al valore contenuto in quell'indirizzo.
4. **Aree di Memoria:** Un programma in esecuzione è suddiviso in diverse zone:
 - **Code Section (Testo):** contiene le istruzioni compilate (spesso in sola lettura).
 - **Data Section:** contiene le variabili globali.
 - **Stack:** gestisce la memoria per le chiamate a funzione (cresce e decresce dinamicamente e automaticamente).
 - **Heap:** area dedicata all'allocazione dinamica della memoria (gestita esplicitamente dal programmatore o dal sistema).

Osservazione critica sulla gestione della memoria: Esistono tre modi principali per gestire la memoria Heap: 1. *Manuale* (come nel C/C++ tramite `new` e `delete`), che garantisce controllo ma alto rischio di memory leak. 2. *Garbage Collection* (come in Java), dove il sistema pulisce automaticamente la memoria non più raggiungibile. 3. *Ownership* (come in Rust), gestita a tempo di compilazione.

1.2 Implementazione del codice

A lezione sono stati presentati vari file C++ per dimostrare visivamente il comportamento della memoria.

1.2.1 Test sulle Variabili (test_vars.cpp)

```
#include <iostream>

int x = 5; // Variabile globale

void f() {
    x = x + 1; // Modifica la variabile globale
    std::cout << "x in f " << x << std::endl;
}

void g() {
    int x = 2, b = 4; // Shadowing: la x locale "nasconde" la x globale
    printf("(x,b) in g: (%d,%d)\n", x, b);
}
```

Commento: In questo snippet osserviamo la differenza tra scope globale e locale. La funzione `f()` modifica la variabile globale `x`. La funzione `g()` dichiara una sua variabile `x` locale: questo fenomeno si chiama *shadowing*. Eventuali modifiche fatte in `g()` non intaccheranno la `x` globale, proteggendo i dati esterni da alterazioni accidentali.

1.2.2 Test sullo Stack Frame (test_stack_frame.cpp)

```
int fact(int n) {
    if (n == 0) return 1;
    else return n * fact(n - 1);
}
```

Commento: Questa semplice funzione ricorsiva genera una pila di *stack frame*. Ad ogni chiamata `fact(n-1)`, un nuovo record di attivazione viene "impilato" sopra il precedente, con il proprio parametro `n`. Quando la condizione base `n == 0` è raggiunta, gli stack frame vengono de-allocati (fase di ritorno), risolvendo l'albero delle moltiplicazioni.

1.2.3 Test sui Puntatori e Modifiche (test_puntatori.cpp)

```
void incrementa_ptr(int* px) {
    *px = *px + 1;
}
```

```

void test_puntatori2() {
    int x = 1;
    incrementa_ptr(&x); // Passaggio per indirizzo
                        // (riferimento simulato in C)
    std::cout << x << std::endl; // Stampa 2
}

```

Commento: Passare una variabile "per valore" significa passarne una copia; le modifiche fatte non persistono. Passando invece l'indirizzo della variabile in memoria tramite un puntatore (&x), la funzione `incrementa_ptr` può dereferenziare il puntatore (`*px`) e alterare il valore originale. Questo concetto sarà vitale in Java, dove gli oggetti vengono sempre gestiti tramite riferimenti.

1.2.4 Test sull'Allocazione Dinamica (test_memoria.cpp)

```

void test_mem_dyn() {
    int* px;
    px = new int;      // Allocazione dinamica sulla Heap
    *px = 3;
    std::cout << *px << std::endl;
    delete(px);       // De-allocazione manuale necessaria nel C++
}

```

Commento: La variabile puntatore `px` risiede nello *Stack* (poiché è locale alla funzione), ma punta a un'area di memoria creata nella *Heap* tramite la keyword `new`. Nel C++, il programmatore è responsabile di distruggere l'allocazione tramite `delete(px)`, altrimenti si incorre in memory leak. Nel corso vedremo come Java ci sollevi da questa responsabilità grazie al suo *Garbage Collector*.

1.3 Esercizi e approfondimenti dal tutorato

Il **Tutorato 1** si concentra fin da subito sull'introduzione al linguaggio Java (sintassi base come vettori e array per calcolare media, mediana e moda) e sui primissimi concetti di programmazione ad oggetti (creazione di classi `Auto`, `Microscopio`, ecc.). Poiché la Lezione 01 è focalizzata esclusivamente sul ripasso dell'architettura in C++, i materiali pratici del Tutorato 1 si allineano e verranno integrati a partire dalla **Lezione 02**, quando introdurremo l'approccio *Object-Oriented* nativo in Java.

2 Lezione 02: Introduzione all'Orientamento agli Oggetti

2.1 Spiegazione teorica

In questa lezione passiamo dal paradigma procedurale a quello ****Orientato agli Oggetti (OOP)****. Se nella programmazione procedurale il focus è sulle funzioni che manipolano dati, nella OOP il focus è sulle "entità" che raggruppano sia i dati che il comportamento.

- **Classe:** È il "progetto" o "stampo" (blueprint). Definisce quali dati (**campi** o attributi) e quali azioni (**metodi**) avranno gli oggetti di quel tipo.
- **Oggetto (Istanza):** È l'entità concreta creata in memoria a partire dalla classe. Ogni oggetto ha il proprio stato (valori dei campi).
- **Costruttore:** Un metodo speciale chiamato quando si crea un oggetto con la keyword `new`. Serve a inizializzare lo stato interno. In Java, se non ne scriviamo uno, ne esiste uno "di default" senza parametri.
- **Il riferimento `this`:** È una parola chiave che indica "questo oggetto specifico". Si usa spesso nel costruttore per distinguere i parametri della funzione dai campi della classe (es. `this.x = x`).

Tipi Enumerativi (Enum): Spesso abbiamo bisogno di variabili che possono assumere solo un set ristretto di valori costanti (es. i giorni della settimana o i punti cardinali). In Java, gli `enum` sono tipi di dato che rendono il codice più leggibile e "type-safe" rispetto all'uso di semplici interi o stringhe.

2.2 Implementazione del codice

Vediamo l'evoluzione della classe TNT usata a lezione per simulare un blocco del gioco Minecraft.

2.2.1 Versione 1: Struttura base e Costruttore

In questa fase definiamo solo lo stato (posizione `x`, `y`, `z`).

```
public class TNT {
    public int x, y, z;

    public TNT(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}
```

Commento: I campi sono `public` (accessibili da chiunque, pratica che vedremo essere sconsigliata) e il costruttore usa `this` per assegnare i valori ricevuti ai campi dell'oggetto.

2.2.2 Versione 2 e 3: Aggiunta di comportamento (Metodi)

Aggiungiamo un metodo per far "esplodere" il blocco e gestire lo stato `primed`.

```

public class TNT {
    public int x, y, z;
    public boolean isPrimed;

    public TNT(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;
        this.isPrimed = false;
    }

    public void explode() {
        if (isPrimed) {
            System.out.println("BOOM at " + x + ", " + y + ", " + z);
        } else {
            System.out.println("TNT is not primed yet.");
        }
    }
}

```

Commento: Qui notiamo come il metodo `explode()` utilizzi i dati interni dell'oggetto (`x`, `y`, `z`, `isPrimed`) per decidere cosa fare. Questo è il cuore della OOP: l'oggetto "sa" come comportarsi in base al suo stato.

2.2.3 Uso delle Enum (`Direction.java`)

```

public enum Direction {
    NORTH, SOUTH, EAST, WEST, UP, DOWN
}

```

Commento: Invece di usare numeri (0, 1, 2...) che potrebbero confondere, usiamo nomi chiari. Java garantisce che una variabile di tipo `Direction` possa contenere solo uno di questi sei valori.

2.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 1**, l'approccio OOP è stato applicato alla modellazione di oggetti del mondo reale. Un esempio significativo è la modellazione di un sistema di **Settings**.

2.3.1 Modellazione gerarchica di oggetti (`Settings`)

L'esercizio richiedeva di creare una classe `Settings` che contiene altri oggetti (Composizione) e usa gli `enum`.

```
// Definizione dello stato tramite Enum
public enum Status { CONNECTED, DISCONNECTED, ERROR }
public enum Theme { DARK, LIGHT, SYSTEM }

public class Network {
    private Status status;
    private String ip;

    public Network(Status status, String ip) {
        this.status = status;
        this.ip = ip;
    }
    // Metodi getter...
}

public class Settings {
    private Network network;
    private Theme theme;

    public Settings(Network n, Theme t) {
        this.network = n;
        this.theme = t;
    }
}
```

Analisi dell'esercizio:

- **Astrazione:** Abbiamo diviso il problema "Impostazioni" in sottoproblemi più piccoli (Network, Theme).
- **Invarianti:** Usando gli `enum` per il tema o lo stato della rete, impediamo al programmatore di inserire valori assurdi (es. un tema "BLUE" se non previsto), aumentando la robustezza del software.

3 Lezione 03: Organizzazione del codice e pattern base

3.1 Spiegazione teorica

Man mano che un progetto cresce, scrivere tutto il codice in un unico file diventa insostenibile. In questa lezione esploriamo i meccanismi offerti da Java per organizzare, proteggere e condividere le funzionalità: *packages*, modificatori di visibilità, parole chiave per l'immutabilità e la globalità, fino ad arrivare a un primo *Design Pattern*.

- **Packages:** Sono i "namespace" di Java, corrispondenti fisicamente alle cartelle sul disco. Risolvono i conflitti di nome (es. avere due classi TNT in pacchetti diversi) e organizzano logicamente il progetto. Per usare classi di altri pacchetti si usa la direttiva `import`.
- **Visibilità e Incapsulamento (Information Hiding):** Nascondere i dettagli implementativi all'interno di una classe è fondamentale per evitare alterazioni esterne indesiderate (es. cambiare la vita di un giocatore ignorando l'armatura).
 - `public`: visibile ovunque.
 - `private`: visibile solo all'interno della classe stessa.
 - `protected`: visibile all'interno dello stesso package e alle sottoclassi.
 - *Package-private* (default, nessuna keyword): visibile solo all'interno dello stesso package.

Per leggere o scrivere in modo controllato i campi `private`, si definiscono metodi pubblici chiamati **getter** e **setter**.

- **Immutabilità (final):** Applicata a una variabile o a un campo, la keyword `final` lo rende una *costante*. Una volta assegnato (o al momento della dichiarazione o nel costruttore), il valore non può più cambiare. Questo previene bug legati alla mutazione inattesa dello stato.
- **Globalità (static):** Un campo o metodo `static` appartiene alla *Classe* e non alla singola istanza (oggetto). Di conseguenza:
 - Esiste una sola copia in memoria per tutta l'esecuzione.
 - Vi si accede usando il nome della classe (es. `Classe.metodo()`).
 - Non si può usare `this` all'interno di metodi `static`.
 - Utile per definire costanti globali (es. `public static final int MAX_STACK = 64`).
- **Design Pattern - Singleton:** I Design Pattern sono soluzioni standard e testate a problemi ricorrenti. Il **Singleton** garantisce che esista *una sola istanza* di una classe nell'intero programma (utile ad esempio per i Settings o oggetti unici del gioco, come il Dragon Egg).

3.2 Implementazione del codice

```
package lecture03.ackages.entities;

public class Player {
    public String username;           // Leggibile e modificabile da tutti
    private int health = 20;          // Nascosto, accessibile solo
                                      // ai metodi interni

    private boolean isPoisoned = false;

    // Setter per controllare in modo sicuro lo stato
    public void setPoisoned(boolean p) {
        this.isPoisoned = p;
    }

    // Getter per leggere lo stato in sola lettura
    public String getUsername() {
        return this.username;
    }

    public void damage(int amount) {
        this.health = this.health - amount;
        if (this.health <= 0) {
            System.out.println("Player died!");
        }
    }
}
```

Commento: Il campo `health` è `private`. Nessun'altra classe può fare `player.health = 0`, "barando" e aggirando la logica di danno del gioco. L'unico modo per sottrarre vita è tramite il metodo pubblico `damage(int amount)`. Questo è l'**Information Hiding**.

3.2.1 Immutabilità con `final` (ToolTier.java)

```
package lecture03.final_mod;

public enum ToolTier {
    WOOD(59, 2.0f), DIAMOND(1561, 8.0f);

    private final int maxUses;
    private final float efficiency;

    ToolTier(int maxUses, float efficiency) {
        this.maxUses = maxUses;
        this.efficiency = efficiency;
    }

    public float getEfficiency() { return this.efficiency; }
}
```

Commento: La velocità (*efficiency*) del diamante deve essere sempre 8.0f. Rendendola *final*, garantiamo a livello di compilatore che non possa essere alterata accidentalmente a runtime.

3.2.2 Variabili di gioco globali e static (GameConstants.java)

```
package lecture03.static_globals;

public class GameConstants {
    // Un valore condiviso da tutto il server
    public static final int MAX_STACK_SIZE = 64;

    // Metodo Utility statico: non ha bisogno dello stato di un oggetto
    public static void printMOTD() {
        System.out.println(">> Welcome to the Minecraft Course!");
    }
}
```

Commento: *MAX_STACK_SIZE* è *static*, quindi è condivisa; non ne viene creata una copia in memoria per ogni blocco o inventario esistente. Poiché è anche *final*, nessuno può modificarne il valore. Si richiamerà con *GameConstants.MAX_STACK_SIZE*.

3.2.3 Il Singleton Pattern (DragonEgg.java)

```
package lecture03.static_globals;

public class DragonEgg {
    // 1. Unica istanza statica, creata al momento del caricamento
    private static DragonEgg THE_INSTANCE = new DragonEgg();

    // 2. Costruttore privato (nessuno può fare 'new DragonEgg()' fuori da qui)
    private DragonEgg() {}

    // 3. Getter pubblico e statico per accedere all'unica istanza
    public static DragonEgg getInstance() {
        return THE_INSTANCE;
    }

    public void teleport() {
        System.out.println(">> Vwoop! The Dragon Egg teleported away.");
    }
}
```

Commento: Questo pattern ruota attorno all'uso combinato di *private* e *static*. Rendendo il costruttore privato, blocchiamo la creazione di nuovi oggetti. Offriamo invece l'accesso alla singola istanza globale tramite il metodo *getInstance()*. Questo assicura, ad esempio, che non si duplichino accidentalmente le risorse o gli asset unici del programma.

3.3 Esercizi e approfondimenti dal tutorato

Anche se il materiale di questa lezione verte in modo massiccio sull'architettura e l'information hiding, il **Tutorato 1** rafforza questi concetti. Negli esercizi proposti dai tutor, la progettazione corretta dei metodi `public` contro l'accesso diretto ai campi `private` risulta essere il "cuore" delle correzioni. Progettare fin da subito pensando a "chi" deve leggere o modificare "cosa" (ed esponendo di conseguenza solo i metodi necessari) è l'approccio ingegneristico che accompagnerà tutto il corso.

4 Lezione 04: Incapsulamento, Invarianti e Memory Layout

In questa lezione si approfondiscono le conseguenze dell'incapsulamento, si introduce il concetto di **invariante** e si analizza in dettaglio come Java gestisce la memoria "dietro le quinte", in particolare il passaggio dei parametri e il layout di classi e oggetti (V-Table).

- **Incapsulamento e Invarianti:** L'incapsulamento non serve solo a nascondere le informazioni, ma è lo strumento principale per garantire gli *invarianti*. Un invariante è una proprietà o una condizione di un oggetto che deve rimanere **sempre vera** durante tutta la sua esistenza. Ad esempio: la vita di un giocatore non può essere negativa, oppure il fuso di una TNT non può essere modificato con valori arbitrari (es. -100). Per far rispettare questi invarianti, si rendono i campi `private` e si gestisce la logica di mutazione unicamente attraverso metodi di classe controllati.
- **Passaggio dei valori (Value vs Reference):** Quando si passa una variabile a un metodo, il comportamento dipende dal tipo di dato:
 - **Tipi Primitivi** (`int`, `double`, `boolean`...): Vengono allocati sullo *Stack* e passati per **Copia** (Passaggio per valore). Le modifiche fatte dal metodo chiamato non si riflettono sulla variabile originale.
 - **Oggetti e Array:** Vengono allocati nella *Heap*. La variabile nello *Stack* contiene in realtà un **Riferimento** (un puntatore) all'oggetto. Quando si passa un oggetto a un metodo, si copia il *riferimento*. Di conseguenza, le modifiche fatte allo stato dell'oggetto all'interno del metodo si rifletteranno anche all'esterno, poiché entrambe le variabili puntano alla stessa area di memoria.
- **Memory Layout e V-Table:** Quando il programma viene caricato, il codice dei metodi (`static` e non) va nella *Code Section*, mentre i campi `static` vanno in un'area *Read-Only*. Ma come fa un oggetto a sapere quali metodi può eseguire? Tramite la **V-Table** (Virtual Table).
 - Ogni classe possiede una V-Table, ovvero un array di puntatori che indicano l'indirizzo in memoria delle implementazioni dei suoi metodi.
 - Ogni oggetto istanziato nella *Heap* ha una struttura precisa: la prima parola (word) di memoria è un puntatore nascosto alla V-Table della sua classe. Le parole successive contengono i valori dei suoi campi (stato).
 - Quando si invoca un metodo su un oggetto (`p.damage()`), Java segue il puntatore alla V-Table, cerca l'offset del metodo e ne esegue il codice corrispondente.

4.1 Implementazione del codice

4.1.1 Protezione degli Invarianti

```
public static void invariants() {
    TNT standardTNT = new TNT();

    // Se fuseLength fosse public, potremmo violare l'invariante del gioco:
    // standardTNT.fuseLength = -100; // ORA E' UN ERRORE DI COMPILAZIONE!

    // Utilizzo corretto e sicuro tramite i metodi previsti:
    standardTNT.ignite();
    standardTNT.tick();
}
```

Commento: Rendendo `fuseLength` e `isIgnited` privati, costringiamo chi usa la classe `TNT` a invocare i metodi `ignite()` e `tick()`, che al loro interno contengono i controlli di sicurezza (es. non far scendere il fuso sotto lo zero, non esplodere se non innescata).

4.1.2 Passaggio dei valori per riferimento e per copia

```
private static void valuePassingExample(){
    int x = 0;
    helper(x);
    // x rimane 0. È stato passato il "valore" 0 copiandolo nel nuovo stack frame.

    TNT t = new TNT();
    reference(t);
    // Se "reference()" modifica lo stato di 't', lo vedremo anche qui.
    // Viene passato l'indirizzo di memoria dell'oggetto.

    TNT[] arrT = new TNT[5];
    // In questo momento abbiamo creato solo "lo spazio" per l'array.
    // Le celle contengono 'null' (il valore di default per i riferimenti).

    for (int i = 0 ; i < 5 ; i++){
        arrT[i] = new TNT(); // Ora creiamo concretamente gli oggetti nella Heap
    }
}
```

Commento: L'errore più comune nei linguaggi ad oggetti è la cosiddetta eccezione di `NullPointerException`. Questo avviene quando si cerca di invocare un metodo o leggere un campo su una variabile riferimento che non punta ad alcun oggetto allocato (come le celle di `arrT` prima del ciclo `for`).

4.1.3 Layout degli oggetti e memoria

```
private static void layoutTest(){
    Player p1 = new Player();
    Player p2 = new Player();

    p2.username = "steve";
    p1.setPoisoned(true);
}
```

Commento (Analisi del Runtime): Quando viene eseguito questo codice: 1. Nello *Stack* del *main* vengono create due variabili, *p1* e *p2*. 2. Nella *Heap* vengono allocati due blocchi di memoria separati. 3. Il layout di *p1* nella memoria *Heap* sarà: - Indirizzo: 0x00A000 (Puntatore alla V-Table di *Player*) - Indirizzo: 0x00A004 (null, valore di default di *username*) - Indirizzo: 0x00A008 (20, valore di default di *health*) - Indirizzo: 0x00A00C (true, perché modificato da *setPoisoned*)

4.2 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 2**, l'esercizio *TestMemoria.java* esplora proprio questi concetti chiedendo di prevedere il comportamento delle variabili. In particolare, si è posta l'enfasi sulla differenza fra l'operatore `==` e le modifiche ai riferimenti.

Quando scriviamo `Auto a = new Auto("SUV"); Auto b = a;`, non stiamo creando due automobili. *a* e *b* sono due variabili sullo *Stack* che puntano all'**esatto stesso indirizzo** nella *Heap*. Di conseguenza, modificando `b.colore = "Rosso";`, vedremo la modifica anche interrogando l'oggetto tramite *a*. Questo fenomeno è alla base dei bug da *side-effect* non previsti, ed è il motivo per cui l'uso di `private` e metodi `final` è cruciale nelle architetture "in the large".

5 Lezione 05: Ereditarietà e Polimorfismo di Sottotipo

5.1 Spiegazione teorica

L'ereditarietà è uno dei pilastri fondamentali della programmazione orientata agli oggetti. Permette di definire una nuova classe (sottoclasse) a partire da una classe esistente (superclasse), ereditandone campi e metodi.

- **Relazione "is-a":** Una sottoclasse è *un* tipo specifico della superclasse (es. un *Cane* è un *Animale*). Questo permette il riutilizzo del codice e la creazione di gerarchie logiche.
- **Keyword extends:** In Java si usa per stabilire il legame di ereditarietà. Una classe può estendere **una sola** altra classe (ereditarietà singola).
- **Polimorfismo di Sottotipo:** È la capacità di un riferimento di tipo superclasse di puntare a un oggetto di una sottoclasse. Ad esempio, una variabile di tipo *Entity* può contenere un oggetto *Player*. Questo è fondamentale per scrivere codice generico che funzioni con intere gerarchie.

- **Costruttori e `super()`:** I costruttori non vengono ereditati. La sottoclasse deve chiamare il costruttore della superclasse tramite la keyword `super(...)` come prima istruzione del proprio costruttore. Se non specificato, Java inserisce automaticamente una chiamata a `super()` senza parametri.
- **La classe `Object`:** In Java, ogni classe che non specifica un genitore estende implicitamente `java.lang.Object`. Essa è la radice di ogni gerarchia e fornisce metodi universali come `toString()`, `equals()` e `hashCode()`.
- **Diamond Problem:** Java non permette l'ereditarietà multipla di classi (estendere due classi contemporaneamente) per evitare ambiguità: se due genitori hanno un metodo con la stessa firma ma implementazione diversa, la sottoclasse non saprebbe quale ereditare.

5.2 Implementazione del codice

A lezione è stata presentata la gerarchia delle entità, simile a quella di un motore di gioco come Minecraft.

5.2.1 La Superclasse (`Entity.java`)

```
public class Entity {
    protected double x, y, z; // Visibile alle sottoclassi

    public Entity(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public void move(double dx, double dy, double dz) {
        this.x += dx;
        this.y += dy;
        this.z += dz;
    }
}
```

Commento: L'uso del modificatore `protected` permette alle classi figlie di accedere direttamente alle coordinate, mantenendole però nascoste al resto del programma (incapsulamento parziale).

5.2.2 La Sottoclasse (LivingEntity.java)

```
public class LivingEntity extends Entity {
    private int health;

    public LivingEntity(double x, double y, double z, int health) {
        super(x, y, z); // Chiamata obbligatoria al costruttore del genitore
        this.health = health;
    }

    public void takeDamage(int amount) {
        this.health -= amount;
        if (this.health <= 0) {
            System.out.println("Entity is dead.");
        }
    }
}
```

Commento: LivingEntity aggiunge lo stato `health` e il comportamento `takeDamage`, ma possiede anche le coordinate e il metodo `move` ereditati da `Entity`.

5.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 2**, l'ereditarietà è stata applicata alla gestione di un inventario tramite una gerarchia di oggetti (`Item`).

5.3.1 Gerarchia degli Oggetti (es5)

L'esercizio prevedeva una classe base `Item` e specializzazioni come `Arma` e `Pozione`.

```
// Superclasse generica
public class Item {
    private String nome;
    public Item(String nome) { this.nome = nome; }
}

// Sottoclasse specializzata
public class Arma extends Item {
    private int danno;
    public Arma(String nome, int danno) {
        super(nome); // Inizializza la parte "Item" dell'oggetto
        this.danno = danno;
    }
}
```

Analisi del Polimorfismo (dal main del tutorato):

```
Item spada = new Arma("Excalibur", 50); // Lecito per polimorfismo
Item pozione = new Pozione("Cura", 20); // Lecito per polimorfismo
```

```
Item[] inventario = {spada, pozione}; // Array polimorfico
```

Osservazione: Grazie al polimorfismo di sottotipo, possiamo gestire un array di `Item` che contiene in realtà oggetti diversi (`Arma`, `Pozione`). Tuttavia, agendo tramite un riferimento di tipo `Item`, non potremo accedere direttamente ai metodi specifici di `Arma` (come un eventuale `getDanno()`) senza un'operazione di casting, che vedremo nelle lezioni successive.

6 Lezione 06: Astrazione e Polimorfismo Ad-hoc

6.1 Spiegazione teorica

L'astrazione è il processo di isolamento delle proprietà essenziali di un'entità, tralasciando i dettagli implementativi che non sono comuni a tutte le sue varianti.

- **Classi Astratte (abstract class):** Una classe dichiarata come astratta non può essere istanziata direttamente (non si può fare `new`). Serve come modello parziale per le sottoclassi. Può contenere sia metodi implementati che metodi astratti.
- **Metodi Astratti (abstract method):** Sono metodi dichiarati senza un corpo (senza implementazione). Rappresentano una "promessa": ogni sottoclasse concreta **deve** fornire un'implementazione per quel metodo.
- **Polimorfismo Ad-hoc:** Si riferisce alla capacità di un metodo di comportarsi in modo diverso in base ai tipi dei parametri o alla classe che lo implementa. Si divide in:
 - **Overloading (Sovraccarico):** Più metodi nella stessa classe con lo **stesso nome** ma **parametri diversi** (per tipo o numero). È risolto a tempo di compilazione (*static polymorphism*).
 - **Overriding (Sovrascrittura):** Una sottoclasse fornisce una nuova implementazione di un metodo già definito nella superclasse (stessa firma). È risolto a runtime (*dynamic polymorphism*).

6.2 Implementazione del codice

6.2.1 Classi e Metodi Astratti (AbstractEntity.java)

```
public abstract class AbstractEntity {
    protected int x, y;

    public AbstractEntity(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // Metodo concreto: tutte le entità si muovono allo stesso modo
    public void moveTo(int newX, int newY) {
        this.x = newX;
        this.y = newY;
    }

    // Metodo astratto: ogni entità ha un modo diverso di "esistere" o agire
    public abstract void update();
}
```

Commento: AbstractEntity definisce la struttura comune (coordinate) e un comportamento condiviso (moveTo), ma delega il comportamento specifico (update) alle classi figlie.

6.2.2 Overloading vs Overriding (OverloadTest.java)

```
public class Calculator {
    // OVERLOADING: stesso nome, parametri diversi
    public int sum(int a, int b) { return a + b; }
    public double sum(double a, double b) { return a + b; }
}

public class SpecialCalculator extends Calculator {
    // OVERRIDING: stessa firma della superclasse, comportamento diverso
    @Override
    public int sum(int a, int b) {
        System.out.println("Using special sum!");
        return a + b + 1;
    }
}
```

Commento: L'overloading permette di gestire tipi diversi con lo stesso nome logico. L'overriding, segnalato dall'annotazione @Override, permette di specializzare un comportamento ereditato.

6.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 3**, il concetto di astrazione viene applicato all'esercizio Flatland.

6.3.1 Esercizio: La gerarchia Flatland

L'obiettivo è modellare abitanti di diverse dimensioni (0D, 1D, 2D) che condividono una base comune.

```
// Superclasse astratta che definisce il concetto di abitante
public abstract class Flatlander {
    private String nome;
    public Flatlander(String nome) { this.nome = nome; }

    // Ogni abitante ha un modo diverso di calcolare la propria "misura"
    public abstract double getMeasure();
}

public class Linelander extends Flatlander {
    private double length;
    public Linelander(String nome, double length) {
        super(nome);
        this.length = length;
    }
    @Override
    public double getMeasure() { return length; }
}
```

Analisi critica: L'uso di `abstract` impedisce di creare un "abitante generico" che non avrebbe senso nella logica del programma[cite: 13, 16]. Obbligando all'overriding di `getMeasure()`, il compilatore garantisce che ogni nuova forma aggiunta al sistema (es. `Squarelander`) implementi correttamente la propria logica di calcolo.

7 Lezione 07: Interfacce e Capacità

7.1 Spiegazione teorica

Se le classi astratte rappresentano "cosa un oggetto è" (un'entità, un animale), le interfacce rappresentano "cosa un oggetto sa fare" (una capacità). In Java, un'interfaccia è un contratto puramente formale.

- **Definizione:** Un'interfaccia può contenere solo firme di metodi (di default pubblici e astratti) e costanti (`public static final`). Non può avere campi di istanza (stato) né costruttori.
- **Ereditarietà Multipla di Tipo:** A differenza delle classi, una classe può implementare **infinite interfacce** (`implements A, B, C...`). Questo permette a un oggetto di assumere molteplici "ruoli" polimorfici.

- **Metodi default (Java 8+):** Introdotti per permettere l'evoluzione delle interfacce senza rompere il codice esistente. Consentono di scrivere un'implementazione standard all'interno dell'interfaccia stessa. Se una classe eredita due metodi **default** identici da due interfacce diverse, Java obbliga il programmatore a fare l'override per risolvere l'ambiguità.
- **Interfacce vs Classi Astratte:**
 - *Classe Astratta:* Si usa per condividere codice e stato tra classi strettamente correlate (relazione forte "is-a").
 - *Interfaccia:* Si usa per definire un protocollo di comunicazione o una capacità comune a classi anche molto diverse tra loro (relazione debole "can-do").

7.2 Implementazione del codice

Vediamo come definire un'interfaccia per oggetti che possono essere "annullati" (come un'esplosione o un comando).

7.2.1 Definizione di un'interfaccia (Cancellable.java)

```
public interface Cancellable {
    // Un campo in un'interfaccia è implicitamente public static final
    int MAX_DELAY = 100;

    // Un metodo è implicitamente public e abstract
    void cancel();

    // Metodo default: fornisce un'implementazione base
    default void logCancel() {
        System.out.println("Operation was cancelled.");
    }
}
```

7.2.2 Implementazione multipla

```
public class ExplodingTNT extends TNT implements Cancellable, Runnable {
    @Override
    public void cancel() {
        this.isPrimed = false;
        System.out.println("Explosion averted.");
    }

    @Override
    public void run() {
        this.explode();
    }
}
```

Commento: ExplodingTNT eredita lo stato da TNT, ma acquisisce la capacità di essere annullata (Cancellable) e di essere eseguita come un thread (Runnable).

7.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 3**, il concetto di interfaccia è centrale nell'esercizio della **Macchinetta delle bevande** e dei sistemi di pagamento.

7.3.1 Sistemi di pagamento polimorfici

L'esercizio richiede di gestire diverse modalità di pagamento (**Moneta**, **CartaDiCredito**, **AppPagamento**) che condividono la capacità di fornire un credito.

```
// Interfaccia che definisce la capacità di pagare
public interface Credito {
    double getValore();
    boolean isValido();
}

public class Moneta implements Credito {
    private double valore;
    public Moneta(double valore) { this.valore = valore; }
    @Override
    public double getValore() { return valore; }
    @Override
    public boolean isValido() { return true; }
}
```

Il Pattern Null Object (Approfondimento dal tutorato): Spesso, se non c'è credito, si rischierebbe un `NullPointerException`. Il tutorato introduce `NullCredito.java`, una classe che implementa `Credito` ma restituisce valore 0.

```
public class NullCredito implements Credito {
    @Override
    public double getValore() { return 0.0; }
    @Override
    public boolean isValido() { return false; }
}
```

Vantaggio: La macchinetta può chiamare `credito.getValore()` senza controllare se l'oggetto è null, rendendo il codice più pulito e robusto.

8 Lezione 08: Composizione vs Ereditarietà e lo Strategy Pattern

8.1 Spiegazione teorica

Finora abbiamo utilizzato l'ereditarietà per condividere stato e comportamento, modellando la relazione *"is-a"* (è un). Tuttavia, l'ereditarietà in Java è singola (una classe può avere una sola superclasse) e intrinsecamente rigida.

- **I limiti dell'Ereditarietà:** Se volessimo modellare un **Piglin** che può attaccare con la spada, con la balestra o a mani nude, potremmo pensare di creare le sotto-classi **SwordPiglin**, **RangedPiglin** e **PunchPiglin**. Ma cosa succederebbe se a un **RangedPiglin** si rompesse la balestra a runtime? L'ereditarietà non ci permette di cambiare la classe di un oggetto durante l'esecuzione: un **RangedPiglin** rimarrebbe tale per sempre. Questo genera un'esplosione combinatoria di classi e una fortissima rigidità.
- **La Composizione ("has-a"):** La soluzione è favorire la *composizione* rispetto all'ereditarietà. Invece di dire "il **Piglin** è **un** tiratore", diciamo "il **Piglin** **ha un** metodo di attacco". L'oggetto principale viene composto da sotto-oggetti che ne definiscono il comportamento.
- **Lo Strategy Pattern:** È un design pattern comportamentale basato sulla composizione che permette di definire una famiglia di algoritmi (strategie), incapsularli in classi separate e renderli intercambiabili.
 - Si definisce un'**Interfaccia** comune per le strategie (es. **AttackStrategy**).
 - Si implementano le **Strategie Concrete** che rispettano il contratto (es. **PunchAttack**, **CrossbowAttack**).
 - La classe di **Contesto** (es. **Piglin**) mantiene un riferimento all'interfaccia e delega l'azione al sotto-oggetto, potendolo sostituire a runtime.

8.2 Implementazione del codice

Vediamo come il **Piglin** utilizzi lo Strategy Pattern per cambiare tipo di attacco dinamicamente senza mai dover cambiare la propria natura (classe).

8.2.1 L'Interfaccia Strategy (AttackStrategy.java)

```
package lecture08.strategy.strategies;
```

```
public interface AttackStrategy {
    // Contratto: tutte le strategie di attacco devono poter essere eseguite
    void execute();
}
```

Commento: Qualsiasi classe che implementa questa interfaccia dovrà fornire la propria logica per il metodo `execute()`.

8.2.2 Le Strategie Concrete

```
// Strategia 1: Attacco corpo a corpo
public class PunchAttack implements AttackStrategy {
    @Override
    public void execute() {
        System.out.println(">> Attack: *Punch* (Melee Hit)");
    }
}

// Strategia 2: Attacco a distanza
public class CrossbowAttack implements AttackStrategy {
    @Override
    public void execute() {
        System.out.println(">> Attack: *TWANG* (Fired Arrow)");
    }
}
```

Commento: La logica dell'attacco non è più cablata nel `Piglin`, ma è separata in classi altamente modulari. Se volessimo aggiungere un `AxeAttack`, basterebbe creare una nuova classe senza toccare nulla del codice esistente (Principio Open/Closed).

8.2.3 La Classe di Contesto (Piglin.java)

```
package lecture08.strategy;
import lecture08.strategy.strategies.*;

public class Piglin {
    // Relazione HAS-A: Il Piglin ha una strategia, non sa quale sia di preciso
    private AttackStrategy currentStrategy;

    public Piglin(){
        // Strategia di default alla creazione
        this.currentStrategy = new PunchAttack();
    }

    // Permette di cambiare la strategia A RUNTIME
    public void setStrategy(AccessStrategy newStrategy) {
        this.currentStrategy = newStrategy;
        System.out.println("Piglin changed strategy to "
                           + newStrategy.getClass().getSimpleName());
    }

    public void fight() {
        if (currentStrategy == null) {
            System.out.println("Piglin stands still (No Strategy).");
            return;
        }
        // DELEGAZIONE: Il Piglin non sa come fare danno, delega all'oggetto
        System.out.print("Piglin acts: ");
        currentStrategy.execute();
    }
}
```

Commento: Questo è il fulcro del pattern. Il metodo `fight()` non contiene `if/else` per capire che arma ha il Piglin, ma fa una semplice *delegazione* al sotto-oggetto tramite `currentStrategy.execute()`.

8.2.4 Modifica del comportamento a Runtime (Lecture8.main)

```
public static void strategyPatternExample() {
    Piglin monster = new Piglin();

    System.out.println("\n--- Round 1: Default Behavior ---");
    monster.fight(); // Eseguirà il PunchAttack

    System.out.println("\n--- Round 2: Picking up a crossbow ---");
    // Sostituiamo il sotto-oggetto: il Piglin cambia comportamento!
    monster.setStrategy(new CrossbowAttack());
    monster.fight(); // Eseguirà il CrossbowAttack
}
```

Commento: Se l'arma del Piglin si rompesse, basterebbe chiamare `monster.setStrategy(new PunchAttack())` e il comportamento ritornerebbe immediatamente quello corpo a corpo. L'ereditarietà non ci avrebbe mai permesso una simile flessibilità dinamica.

9 Lezione 09 (Lab 1): Modellazione, Identità degli oggetti e gerarchie

9.1 Spiegazione teorica e Struttura del Laboratorio

Questa lezione rappresenta il primo di una serie di laboratori pratici. Nei laboratori, l'obiettivo si sposta dalla teoria alla *modellazione attiva* di un problema, per cominciare a prepararsi ai temi d'esame.

Il focus del Laboratorio 1 è il primo passo dell'Ingegneria del Software orientata agli oggetti. Data una specifica testuale, si deve procedere per fasi logiche:

- **Identificare le Classi e i Campi (Fase 1):** Quali entità diventano classi? Quali informazioni costituiscono lo "stato" di queste entità?
- **Modificatori di visibilità (Fase 2):** Quali campi devono essere `private` per garantire l'incapsulamento e gli invarianti? Quali metodi devono essere `public` per offrire un'interfaccia usabile?
- **Uso di `final` e `static` (Fase 3):** Quali informazioni non cambiano mai nel ciclo di vita dell'oggetto (`final`)? Quali appartengono alla classe in generale e non alla singola istanza (`static`)?
- **Identificazione delle Enum (Fase 4):** Quali concetti rappresentano un set finito di opzioni categoriche (anziché usare "magic numbers" o stringhe) e richiedono un'enumerazione?

Durante i laboratori, si discute in gruppo non tanto per scrivere subito l'algoritmo perfetto, ma per definire le *firme*, le gerarchie e lo *scheletro* architetturale.

9.2 Implementazione pratica: Esercizio di Modellazione

Simulando il lavoro di gruppo del Lab 1, proviamo a modellare l'identità di un'arma in un gioco:

```
// Fase 4: Identificazione delle Enum
public enum Rarity {
    COMMON, UNCOMMON, RARE, EPIC, LEGENDARY
}

public class Weapon {
    // Fase 3: Scelta dei modificatori final e static
    public static final int MAX_DURABILITY = 100; // Globale a tutte le armi

    // Fase 1 e 2: Definizione dei campi e della visibilità
    private final String name;           // Immutabile e nascosto
    private final Rarity rarity;          // Immutabile e nascosto
    private int durability;               // Mutevole, ma privato (Incapsulamento)

    public Weapon(String name, Rarity rarity) {
        this.name = name;
        this.rarity = rarity;
        this.durability = MAX_DURABILITY;
    }

    // API per manipolare lo stato in modo sicuro mantenendo gli invarianti
    public void use() {
        if (this.durability > 0) {
            this.durability--;
            System.out.println(this.name+"used. Durability:"+this.durability);
        } else {
            System.out.println(this.name + " is broken!");
        }
    }
}
```

Commento e Analisi: Questo snippet riassume l'obiettivo formativo del Lab 1. La classe `Weapon` nasconde il suo stato interno (`private`). Il nome e la rarità sono `final` perché l'identità dell'arma non cambia nel tempo. L'uso della `enum Rarity` scongiura l'uso di stringhe vulnerabili a errori di battitura. Infine, `MAX_DURABILITY` è stato reso `static` e `final` perché è una regola globale del gioco.

9.3 Esercizi e approfondimenti dal tutorato

I concetti dei laboratori si intrecciano fortemente con gli esercizi assegnati nei tutorati. Un ottimo esempio affrontato dai tutor (dal **Tutorato 1 e 2**) per modellare l'identità e i vincoli degli oggetti è l'esercizio **Microscopio e Batterio**.

Nell'esercizio viene chiesto di estrarre dal testo le classi e la loro struttura:

- **Le Classi:** Batterio, Microscopio.
- **Le Enum:** Morfologia (BACILLO, COCCO, SPIRILLO) e TipoMicroscopio (OTTICO, ELETTRONICO).
- **L'Identità:** Un batterio possiede un nome (`final`) e una morfologia (`final`), modellati con un costruttore mirato.

Questo approccio metodico alla lettura del testo — estraendo prima i *sostantivi* (potenziali classi o enum) e poi definendone i *vincoli* (`static`, `final`, `private`) — è il cuore pulsante del *programming in the large* e la chiave per affrontare correttamente i successivi temi d'esame.

10 Lezione 10: Il Dispatch (Static vs Dynamic Binding) e la risoluzione degli argomenti

10.1 Spiegazione teorica

In Java, il processo che determina quale blocco di codice eseguire quando viene invocato un metodo si chiama **Dispatch** (o Binding). Per capirlo a fondo, dobbiamo distinguere due concetti fondamentali legati alle variabili:

- **Tipo Statico (Compile-time type):** È il tipo con cui la variabile è stata dichiarata nel codice (es. `Block b`). È l'unico tipo che il compilatore "vede". Determina *quali* metodi è possibile chiamare su quell'oggetto.
- **Tipo Dinamico (Runtime type):** È il tipo dell'oggetto reale che è stato allocato in memoria nella Heap tramite la keyword `new` (es. `= new DiamondBlock()`). Determina *quale implementazione* del metodo verrà effettivamente eseguita a runtime.

A partire da questa distinzione, Java applica due meccanismi di binding:

1. **Dynamic Binding (Late Binding / Dynamic Dispatch):** È il comportamento standard in Java. Quando si invoca un metodo su un oggetto, la Java Virtual Machine guarda il **tipo dinamico** dell'oggetto per decidere quale codice eseguire. Se la sottoclasse ha fatto l'overriding del metodo, verrà eseguita l'implementazione della sottoclasse.
2. **Static Binding (Early Binding / Static Dispatch):** Avviene a tempo di compilazione. Il compilatore collega la chiamata al metodo usando il **tipo statico** dell'oggetto. Questo accade obbligatoriamente (e unicamente) per i metodi dichiarati `final`, `static` e `private`, perché, per definizione, tali metodi non possono subire overriding.

Interazione tra Dispatch e Overloading (Risoluzione degli argomenti): L'overloading (stesso nome del metodo, parametri diversi) viene risolto **a tempo di compilazione**. Il compilatore decide quale variante (firma) del metodo chiamare basandosi esclusivamente sul **tipo statico** degli argomenti passati. Il *Dynamic Dispatch* si applica solo all'oggetto ricevente (quello alla sinistra del punto), mai ai suoi argomenti.

10.2 Implementazione del codice

10.2.1 Static vs Dynamic Binding (Block e DiamondBlock)

```
public class Block {
    // Metodo normale: soggetto a Dynamic Binding
    public void onBreak() {
        System.out.println("Generic Block breaks into pieces.");
    }

    // Metodo final: soggetto a Static Binding
    public final void getRegistryName() {
        System.out.println("ID: minecraft:generic_block");
    }
}

public class DiamondBlock extends Block {
    @Override
    public final void onBreak() {
        System.out.println("DiamondBlock: *CLING* (Drops Diamonds!)");
    }
    // L'override di getRegistryName() genererebbe un errore di compilazione
}
```

Test del Binding:

```
private static void predictBindingExample() {
    Block bd = new DiamondBlock();
    // Tipo statico di bd: Block
    // Tipo dinamico di bd: DiamondBlock

    bd.onBreak();
    // Dynamic Binding: si valuta il tipo dinamico (DiamondBlock).
    // Stampa: "DiamondBlock: *CLING* (Drops Diamonds!)"

    bd.getRegistryName();
    // Static Binding (a causa del final): si valuta il tipo statico (Block).
    // Stampa: "ID: minecraft:generic_block"
}
```

Commento: Benché bd contenga un DiamondBlock, il compilatore ci impedisce di invocare metodi esclusivi del diamante (come un ipotetico `getDiamond()`) perché il tipo statico Block non li possiede. L'IDE vi darebbe errore.

10.2.2 La risoluzione degli argomenti (Miner e Pick)

```
public class Miner {
    // Variante 1: ascia normale
    public void mine(Pick p) {
        System.out.println("-> Used Generic Mining (Slow)");
    }

    // Variante 2: ascia in diamante
    public void mine(DiamondPick p) {
        System.out.println("-> Used Diamond Mining (Instant!)");
    }
}

public static void argBindingExample() {
    Miner steve = new Miner();

    Pick hiddenDiamond = new DiamondPick();
    // Tipo statico dell'argomento: Pick
    // Tipo dinamico dell'argomento: DiamondPick

    steve.mine(hiddenDiamond);
}
```

Commento: L'invocazione `steve.mine(hiddenDiamond)` stamperà `-> Used Generic Mining (Slow)`. Perché non ha usato la variante superveloce? Perché il compilatore, nel momento in cui sceglie quale metodo in overloading usare, valuta il **tipo statico** dell'argomento (`Pick`). L'overloading si risolve in modo statico e ignora il reale contenuto dinamico della variabile.

10.3 Esercizi e approfondimenti dal tutorato

Negli esercizi visti a lezione e a tutorato sul binding, la regola d'oro da applicare per non farsi fregare durante gli esami è divisa in due step mentali:

1. **Fase 1 (Compilazione e Staticità):** Guarda il *tipo statico* dell'oggetto che chiama il metodo. Verifica se il metodo esiste lì. Se ci sono parametri in overloading, scegli la firma esatta usando i *tipi statici* degli argomenti passati. Se il metodo non viene trovato, è un *Errore di Compilazione*.
2. **Fase 2 (Esecuzione e Dinamicità):** Se la firma è stata trovata, controlla i modificatori. Se il metodo è `final`, `static` o `private`, esegui la versione trovata nella Fase 1. Altrimenti (metodo virtuale normale), passa al *tipo dinamico* dell'oggetto. Cerca dal basso della gerarchia (partendo dal tipo dinamico) verso l'alto la versione sovrascritta (`@Override`) più specifica per quella firma ed esegui quella.

11 Lezione 11: V-Table, Casting, instanceof e Template Method

11.1 Spiegazione teorica

Questa lezione fa da ponte tra i concetti di basso livello (come la JVM gestisce la memoria per l'ereditarietà) e quelli di alto livello (Design Patterns per evitare codice fragile).

- **La V-Table (Virtual Table):** Abbiamo visto che il *Dynamic Dispatch* sceglie il metodo a runtime in base al tipo dinamico dell'oggetto. Ma come fa fisicamente? Ogni classe possiede una V-Table, ovvero una tabella (array) di puntatori alle locazioni di memoria dove risiedono le istruzioni dei suoi metodi. Quando un oggetto viene allocato nella *Heap*, la sua primissima parola di memoria è un puntatore nascosto alla V-Table della sua classe. Se invochiamo `miner.mine(p)`, Java va all'indirizzo dell'oggetto `miner`, segue il puntatore alla V-Table, salta all'offset corrispondente al metodo `mine` ed esegue il codice.
- **Casting e instanceof:** A volte il tipo statico di una variabile è generico (es. `Entity`), ma abbiamo l'esigenza di invocare un metodo specifico della sua classe reale (es. `hiss()` di `Creeper`).
 - **instanceof:** È un operatore che restituisce `true` se il tipo dinamico dell'oggetto a runtime è compatibile con la classe specificata.
 - **Casting (Downcasting):** Forza il compilatore a trattare una variabile generica come se fosse di un tipo più specifico (es. `(Creeper) myEntity`). Se ci sbagliamo e l'oggetto non è davvero di quel tipo, il programma va in crash a runtime con una `ClassCastException`.

Attenzione (Code Smell): Il professore sottolinea che l'uso massiccio di `instanceof` e casting è un sintomo di pessima progettazione a oggetti. Viola il Principio Open/Closed (se aggiungo una classe, devo scovare e modificare decine di `if/else` sparsi nel codice), è prone a errori ed è lento. La soluzione è sempre il **Polimorfismo**.

- **Il Pattern Template Method:** È un *Behavioral Pattern* che risolve proprio l'abuso di `instanceof`. Si applica quando diverse classi eseguono la stessa identica sequenza di operazioni (un algoritmo standard), ma differiscono nei dettagli di alcuni passaggi.
 - Si crea una classe astratta base.
 - Si definisce un metodo **scheletro** (il *Template Method*), contrassegnato come `final` affinché l'ordine delle operazioni non possa essere alterato.
 - Lo scheletro richiama altri metodi interni (`abstract` o `protected`) che le sottoclassi implementeranno per fornire il comportamento specifico del singolo step.

11.2 Implementazione del codice

11.2.1 Il problema: L'abuso di instanceof (BadBlock.java)

```
public class BadBlock {
    public void destroyBlock() {
        System.out.println("1. Spawning Particles...");
        System.out.print("2. Playing Sound: ");

        // CATTIVA PRATICA: Controlli manuali sul tipo (lento e fragile)
        if (this instanceof BadGlass) {
            System.out.println("*Tink* (Glass Sound)");
        } else {
            System.out.println("*Crack* (Default Stone Sound)");
        }

        System.out.print("3. Dropping: ");
        if (this instanceof BadDiamond) {
            BadDiamond diamond = (BadDiamond) this; // Casting
            diamond.dropSpecialDiamond();
        } else if (this instanceof BadGlass) {
            System.out.println("Nothing (Shattered)");
        } else {
            System.out.println("Cobblestone");
        }
    }
}
```

Commento: Questo codice è fragile. Se domani aggiungessimo un `BadWoodBlock`, dovremmo aprire questa classe e aggiungere altri `else if`. Le classi non sono indipendenti, violando le regole base dell'OOP.

11.2.2 La soluzione: Il Template Method (AbstractBlock.java)

```
public abstract class AbstractBlock {

    // IL TEMPLATE METHOD: È final, fissa l'algoritmo (lo scheletro)
    public final void destroyBlock() {
        spawnParticles();    // Step 1: Logica privata, fissa per tutti
        playBreakSound();    // Step 2: Sovrascrivibile (facoltativo)
        dropLoot();          // Step 3: Obbligatorio per le sottoclassi
    }

    private void spawnParticles() {
        System.out.println(" *Poof* (Particles spawned)");
    }

    // Comportamento di default che le sottoclassi possono cambiare (Hook)
    protected void playBreakSound() {
        System.out.println(" *Crack* (Sound played)");
    }

    // Comportamento astratto: ogni blocco DEVE dire cosa dropa
    protected abstract void dropLoot();
}
```

Commento: La sequenza delle operazioni (`destroyBlock`) è blindata dal `final`. Le sottoclassi non devono più preoccuparsi di *quando* fare un'azione, ma solo di *come* farla all'interno del proprio contesto.

11.2.3 Le implementazioni concrete

```
public class DiamondOre extends AbstractBlock {
    @Override
    protected void dropLoot() {
        System.out.println(" Dropped: 1x Diamond");
    }
}

public class GlassBlock extends AbstractBlock {
    @Override
    protected void dropLoot() {
        System.out.println(" Dropped: Nothing (It shattered)");
    }

    @Override
    protected void playBreakSound() {
        System.out.println(" *Tink* (Glass Sound)");
    }
}
```

Commento: Le classi figlie sono ora pulite ed estremamente focalizzate solo sulle proprie peculiarità.

11.3 Esercizi e approfondimenti a lezione

Per verificare la flessibilità del pattern, è stato richiesto di espanderlo per gestire i blocchi di legno. La potenza del *Template Method* sta nel permettere di creare livelli intermedi nella gerarchia senza intaccare lo scheletro originale.

```
// Livello intermedio: fissa il suono per TUTTI i legni, ma lascia
// astratto il drop
public abstract class AbstractWoodBlock extends AbstractBlock {
    @Override
    protected void playBreakSound() {
        System.out.println(" (Soft Wood Sound)");
    }
}

// Implementazione finale: l'albero specifico definisce solo il suo loot
public class OakBlock extends AbstractWoodBlock {
    @Override
    protected void dropLoot() {
        System.out.println(" -> Dropped: Oak Log");
    }
}
```

Conclusione: Se volessimo aggiungere un `CherryBlock`, basterebbe estendere `AbstractWoodBlock` e implementare una singola riga per il drop. Il codice madre del distruttore e della gestione del suono non verrebbe toccato, rispettando appieno il principio dell'ereditarietà.

12 Lezione 12 (Lab 2): Classi astratte, Metodi e Refactoring

12.1 Spiegazione teorica e Struttura del Laboratorio

Questo secondo laboratorio sposta l'attenzione dalla semplice identificazione delle entità (vista nel Lab 1) alla **progettazione avanzata delle gerarchie e dei comportamenti**. L'obiettivo è abituarsi a fare scelte architetture consapevoli in base al testo di un problema.

I gruppi di lavoro affrontano le seguenti fasi di modellazione:

- **Fase 1: Le classi astratte:** Non tutte le entità identificate in un testo devono poter essere istanziate direttamente. Capire quando una classe serve solo da "stampo base" e da collettore di comportamenti comuni (diventando quindi **abstract**) è essenziale per evitare l'istanziamento di oggetti logici incompleti o troppo generici.
- **Fase 2: I metodi, astratti e non:** All'interno di una classe astratta o di un genitore, il programmatore deve tracciare un confine netto: quali comportamenti hanno un'implementazione universale e condivisa da tutti i figli (metodi *concreti*), e quali invece rappresentano solo un contratto che *deve* essere specializzato da ogni figlio (metodi *astratti*).

- **Fase 3: Campi che dovrebbero essere metodi:** Un errore gravissimo e comunissimo nella progettazione è salvare come *stato* (in una variabile) un'informazione che è in realtà **derivabile** o calcolabile al volo da altri stati. Trasformare questi campi in metodi previene inconsistenze logiche.

12.2 Implementazione pratica: Esercizio di Refactoring

Per illustrare il cuore della "Fase 3", consideriamo un errore classico di modellazione e la sua risoluzione ingegneristica.

```
// MODELLAZIONE ERRATA: "isAlive" è un campo statico nello stato
public class BadPlayer {
    private int health = 20;
    private boolean isAlive = true; // Rischio altissimo di inconsistenza!

    public void takeDamage(int amount) {
        this.health -= amount;
        // Se non ci ricordiamo di aggiornare isAlive, il giocatore
        // avrà vita <= 0 ma risulterà ancora vivo per il resto del gioco.
        if (this.health <= 0) {
            this.isAlive = false;
        }
    }
}
```

```
// MODELLAZIONE CORRETTA: "isAlive" diventa un metodo (Proprietà calcolata)
public class GoodPlayer {
    private int health = 20;

    public void takeDamage(int amount) {
        this.health -= amount;
    }

    // Il campo sparisce e diventa un metodo:
    // valuta lo stato "in tempo reale", garantendo l'invariante logico.
    public boolean isAlive() {
        return this.health > 0;
    }
}
```

Commento e Analisi: Nel primo caso stiamo memorizzando informazioni ridondanti. Mantenere sincronizzati `health` e `isAlive` attraverso decine di metodi (danno, cure, pozioni, respawn) è prone a errori (*bug*). Nel secondo caso delegando l'informazione "vita" a un metodo derivato (`isAlive()`), si rende la classe a prova di errore e si riduce anche la memoria occupata (un `boolean` in meno per ogni oggetto allocato).

12.3 Esercizi e approfondimenti dal tutorato

Il materiale dei tutorati e le correzioni in classe si concentrano moltissimo sulla **scomposizione del polimorfismo**.

Spesso un esame (o un esercizio) si presenta con una richiesta che invoglia a usare uno **switch** o un **if/else** infinito per decidere come comportarsi in base a un tipo (es. "Se è un blocco di legno suona X, se è vetro suona Y"). Il laboratorio insegna a estrarre questa logica creando una **ClasseAstratta** genitore con un metodo **abstract void suona()**, e tante sottoclassi concrete quante sono le varianti. Il programma principale chiamerà semplicemente **blocco.suona()** senza più doversi preoccupare del tipo specifico del blocco, sfruttando in pieno il **Dynamic Dispatch** (visto nella Lezione 10) e ripulendo drasticamente l'architettura.

13 Lezione 13: Gestione degli Errori ed Eccezioni

13.1 Spiegazione teorica

Nei linguaggi meno recenti (come il C), gli errori venivano gestiti tramite "codici di errore" (es. ritornare -1 se un'operazione falliva). Questo approccio porta a numerosi problemi: come differenziare gli errori? Cosa succede se -1 è un valore di ritorno lecito? Come separare la logica di business dalla gestione degli errori?

I linguaggi orientati agli oggetti risolvono questo problema elevandolo a livello semantico tramite le **Eccezioni**.

- **La gerarchia delle eccezioni:** In Java, tutte le eccezioni estendono la classe radice **Throwable**. Da essa derivano due grandi rami:
 - **Error:** Errori critici della JVM (es. **OutOfMemoryError**). Non vanno mai gestiti.
 - **Exception:** Errori recuperabili derivanti dalla logica dell'applicazione. Si dividono a loro volta in:
 - * **Checked Exceptions:** Il compilatore obbliga il programmatore a gestirle (tramite **try-catch** o dichiarandole nella firma del metodo).
 - * **Unchecked Exceptions (RuntimeException):** Eccezioni dovute a bug del programmatore (es. **NullPointerException**, **IndexOutOfBoundsException**). Queste **non** vanno gestite con un **try-catch**, ma prevenute sistemando il codice logico (es. controllando che l'indice non sia fuori dai limiti prima di accedere all'array).

- **Sintassi base (throw e throws):**

- `throw new ...`: Solleva materialmente l'eccezione, interrompendo l'esecuzione normale del metodo.
- `throws ...`: Si mette nella firma del metodo per dichiarare che il metodo "potrebbe" sollevare tali eccezioni, scaricando la responsabilità di gestirle a chi lo invoca.

- **Gestione (try-catch):** Se un metodo dichiara di lanciare un'eccezione checked, chi lo chiama deve racchiuderlo in un blocco `try`. Se l'eccezione si verifica, il flusso salta al blocco `catch` corrispondente. **Regola fondamentale:** i blocchi `catch` vanno ordinati dal più specifico (sottoclasse) al più generico (superclasse).

- **Eccezioni e Subtyping (Overriding):** Se una sottoclasse fa l'override di un metodo, può dichiarare (`throws`) solo le stesse eccezioni del metodo originale, un sottoinsieme di esse, oppure sottotipi di quelle originali. **Non può dichiarare eccezioni nuove o più generiche.** Questo garantisce che il polimorfismo non si rompa a runtime.

- **Try-with-resources:** Un costrutto speciale (es. `try (FileReader f = new FileReader(...)) { ... }`) introdotto per chiudere automaticamente le risorse (file, connessioni) al termine del blocco, anche in caso di errore. Funziona solo con oggetti che implementano l'interfaccia `AutoCloseable`.

13.2 Implementazione del codice

13.2.1 Definizione di una gerarchia di Eccezioni

Creare eccezioni personalizzate è utile per dare semantica precisa agli errori del nostro dominio (es. Minecraft).

```
// Eccezione base del nostro dominio
public class CraftingException extends Exception {
    public CraftingException(String msg) { super(msg); }
}

// Eccezioni specifiche figlie di CraftingException
public class RecipeMissingException extends CraftingException {
    public RecipeMissingException(String msg) { super(msg); }
}

public class InventoryFullException extends CraftingException {
    public InventoryFullException(String msg) { super(msg); }
}
```

13.2.2 Sollevare e dichiarare le eccezioni (CraftingTable.java)

```
public class CraftingTable {
    // La firma dichiara quali errori checked possono sfuggire
    // da questo metodo
    public Block craftAdvanced(String item) throws RecipeMissingException,
                                                InventoryFullException {

        if (item.equals("UnknownThing")) {
            // Generazione materiale dell'errore
            throw new RecipeMissingException("What is a " + item + "?");
        }

        boolean hasSpace = false;
        if (!hasSpace) {
            throw new InventoryFullException("No empty slots available.");
        }
        return new Block("Boat");
    }
}
```

13.2.3 Catturare e gestire gli errori

```
public static void exceptionsExample() {
    CraftingTable table = new CraftingTable();

    try {
        table.craftAdvanced("Diamond_Sword"); // Potrebbe fallire
    }
    catch (RecipeMissingException e) {
        System.out.println("Unknown Recipe: " + e.getMessage());
        System.out.println(">> Opening Recipe Book...");
    }
    catch (InventoryFullException e) {
        System.out.println("Inventory Full: " + e.getMessage());
        System.out.println(">> Please clear a slot.");
    }
    catch (Exception e) { // Catch generico di "salvataggio" sempre alla fine
        System.out.println("Critical Error: " + e.getMessage());
    }
}
```

Commento: L'uso del try-catch divide in modo netto la *business logic* (il tentativo di craftare nel try) dalla *error logic* (i vari catch).

13.2.4 Eccezioni e Costruttori

I costruttori non possono restituire valori, quindi le eccezioni sono **l'unico modo** per segnalare e bloccare la creazione di un oggetto in uno stato invalido.

```
public CraftingTable(int n) throws CraftingException {
    if (n < 0) {
        throw new CraftingException("Cannot create table with negative slots");
    }
}
```

13.3 Esercizi e approfondimenti dai laboratori (Lab 3)

Il **Laboratorio 3** pone l'accento sul processo di ingegnerizzazione degli errori:

1. **Quali eccezioni definire?** Leggendo un tema d'esame, ogni frase che esprime un vincolo ("Un giocatore non può usare un'arma rotta", "Un prelievo non può superare il saldo") si traduce in una classe `Exception` (es. `BrokenWeaponException`, `InsufficientFundsException`).
2. **Dove lanciarle (throw)?** Solitamente all'inizio dei metodi di mutazione dello stato (`setter`, `withdraw`, `attack`) per validare gli input o lo stato interno.
3. **Dove catturarle (try-catch)?** Mai all'interno delle classi di modello base. Le eccezioni vanno "tirate" verso l'alto e gestite nei *Controller* o nell'interfaccia utente (es. la classe `Main` o un `GameEngine`), in modo da poter mostrare un messaggio di errore all'utente.

14 Lezione 14: Modellazione e Spawner

14.1 Spiegazione teorica

In questa lezione il focus si sposta dalla singola classe alla progettazione di un ****sistema di gestione****. Spesso nei software (specialmente nei videogiochi o nei sistemi di monitoraggio) non gestiamo oggetti isolati, ma intere popolazioni di entità che nascono, agiscono e muoiono.

- **Responsibility-Driven Design:** Bisogna decidere "chi fa cosa". Se abbiamo molte entità (come i mostri in Minecraft), non è efficiente che ognuna si gestisca da sola nel programma principale. Serve un oggetto "Manager", chiamato ****Spawner****, che ha la responsabilità di:
 - Mantenere un elenco (collezione) di tutte le entità attive.
 - Creare nuove istanze e aggiungerle alla gestione.
 - Aggiornare lo stato di tutte le entità in un unico ciclo (*tick*).

- **Introduzione alle Collezioni (List):** Per gestire un numero variabile di entità, non usiamo gli array (che hanno dimensione fissa), ma le liste dinamiche di Java, come `ArrayList<T>`.
 - `List<Entity>`: È un primo esempio di **Generics**. Indica una lista che può contenere solo oggetti di tipo `Entity` o sottoclassi.
 - `entities.add(e)`: Aggiunge un elemento alla fine.
 - `for (Entity e : entities)`: Ciclo *for-each* per iterare su tutta la collezione.

14.2 Implementazione del codice

Vediamo il cuore dello `Spawner`, che utilizza il polimorfismo per aggiornare entità diverse senza conoscerne il tipo specifico.

14.2.1 La classe `Spawner` (`Spawner.java`)

```
package lecture14.spawn;
import java.util.ArrayList;
import java.util.List;

public class Spawner {
    // Collezione di entità gestite
    private final List<Entity> entities;

    public Spawner() {
        this.entities = new ArrayList<>();
    }

    // Aggiunge un'entità alla gestione
    public void spawn(Entity e) {
        this.entities.add(e);
    }

    // Aggiornamento globale (Tick)
    public void tick() {
        System.out.println("--- Spawner Tick ---");
        for (Entity e : entities) {
            // Polimorfismo: ogni entità agisce secondo la propria logica
            e.update();
        }
    }
}
```

Commento: Lo `Spawner` non sa se nella lista c'è un `Piglin` o uno `Zombie`. Sa solo che sono tutte `Entity` e che possiedono un metodo `update()`. Questa è la potenza del **Dynamic Binding** applicato alle collezioni.

14.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 4**, il concetto di modellazione di sistemi è stato approfondito con l'esercizio della **Sonda**.

14.3.1 Modellazione di Sistemi Gerarchici (Sonda)

L'esercizio richiedeva di modellare una **Sonda** base e una **SondaAvanzata** che estende le funzionalità di rilevamento.

- **Sonda base:** Gestisce un identificativo e una posizione.
- **SondaAvanzata:** Aggiunge sensori per la temperatura o la pressione.

Analisi del problema: Proprio come lo **Spawner** gestisce diverse entità, un **Satellite** potrebbe gestire una `List<Sonda>`. Questo permette di inviare un comando "rileva dati" a tutte le sonde contemporaneamente, sfruttando l'ereditarietà per far sì che ogni sonda (base o avanzata) risponda con i dati che è in grado di raccogliere.

15 Lezione 15: Creazione ed Evoluzione (Factory Method e State Pattern)

15.1 Spiegazione teorica

Fino a questo momento abbiamo creato gli oggetti usando direttamente la keyword **new** (es. `new Zombie()`). Tuttavia, l'uso diretto di **new** accoppia fortemente (lega) il nostro codice alla classe concreta. Cosa succede se vogliamo che uno **Spawner** generi mostri diversi in base al livello di difficoltà, senza dover riempire il codice di `if/else`? E cosa succede quando un'entità cambia drasticamente comportamento nel tempo (es. un **Creeper** che si innesca)?

Per risolvere queste sfide architetturali, la *Gang of Four (GoF)* ha definito due pattern cruciali:

- **Factory Method Pattern (Creazionale):** Risolve il problema della creazione degli oggetti. Invece di chiamare **new** direttamente, si delega l'istanziatura a un metodo speciale (il *Factory Method*).
 - Permette a una superclasse di definire un'interfaccia per la creazione di un oggetto, ma lascia alle sottoclassi la decisione su *quale* classe concreta istanziare.
 - Favorisce il polimorfismo fin dal momento della nascita dell'oggetto.
- **State Pattern (Comportamentale):** Risolve il problema di un oggetto il cui comportamento cambia in base al suo stato interno.
 - Invece di usare variabili booleane (`isAngry`, `isSleeping`) e riempire i metodi di `if(isAngry) { ... }`, lo stato stesso diventa un **Oggetto** (simile allo *Strategy Pattern*).
 - L'oggetto principale (Contesto) delega l'esecuzione delle azioni all'oggetto "Stato" corrente. Quando lo stato logico cambia, il Contesto sostituisce il proprio oggetto Stato con uno nuovo.

15.2 Implementazione del codice

15.2.1 Il Factory Method (MobSpawner)

Immaginiamo di avere un gioco con diverse zone (Normale e Nether). Vogliamo uno spawner generico, ma che generi mostri appropriati per la zona.

```
// La classe creatrice astratta
public abstract class MobSpawner {
    // IL FACTORY METHOD: delega la creazione alle sottoclassi
    protected abstract Entity createMob();

    // Metodo logico comune che usa il Factory Method
    public void spawnWave() {
        Entity mob = createMob(); // Non so quale mob sia,
                                   // ma so che è un'Entity
        mob.spawnInWorld();
        System.out.println("Spawned a new"+mob.getClass().getSimpleName());
    }
}

// Fabbrica concreta 1
public class OverworldSpawner extends MobSpawner {
    @Override
    protected Entity createMob() {
        return new Zombie(); // Sceglie di creare uno Zombie
    }
}

// Fabbrica concreta 2
public class NetherSpawner extends MobSpawner {
    @Override
    protected Entity createMob() {
        return new Piglin(); // Sceglie di creare un Piglin
    }
}
```

Commento: Se aggiungiamo una nuova dimensione (es. The End), non dobbiamo modificare il codice esistente. Creiamo semplicemente un `EndSpawner` che restituisce un `Enderman`. Principio Open/Closed rispettato perfettamente.

15.2.2 Lo State Pattern (Creeper)

Un Creeper ha due stati: "Idle" (Calmo) e "Ignited" (Innescato). A seconda dello stato, reagisce diversamente al tempo che passa (tick).

```
// 1. L'Interfaccia dello Stato
public interface CreeperState {
    void tick(Creeper context);
}

// 2. Stato Concreto: Calmo
public class IdleState implements CreeperState {
    @Override
    public void tick(Creeper context) {
        System.out.println("Creeper is wandering around aimlessly...");
        // Se il giocatore è vicino, cambia stato!
        if (context.isPlayerNear()) {
            System.out.println("*Hiss* Creeper is ignited!");
            context.setState(new IgnitedState()); // Transizione di stato
        }
    }
}

// 3. Stato Concreto: Innescato
public class IgnitedState implements CreeperState {
    private int fuse = 3;

    @Override
    public void tick(Creeper context) {
        fuse--;
        if (fuse <= 0) {
            System.out.println("BOOM! Creeper exploded!");
            context.die();
        } else {
            System.out.println("Creeper is swelling up... " + fuse);
        }
    }
}
```

```
// 4. Il Contesto (Il Creeper stesso)
public class Creeper extends Entity {
    private CreeperState currentState;

    public Creeper() {
        this.currentState = new IdleState(); // Stato iniziale
    }

    public void setState(CreeperState newState) {
        this.currentState = newState;
    }

    @Override
    public void update() {
        // Delega il comportamento allo stato corrente
        currentState.tick(this);
    }
}
```

Commento: Senza lo State Pattern, il metodo `update()` del Creeper sarebbe un enorme e confuso blocco `if-else`. Con questo pattern, ogni classe di stato si concentra solo sulle proprie regole, rendendo l'evoluzione del comportamento pulita e modulare.

15.3 Esercizi e approfondimenti dal tutorato

La gestione degli stati è un concetto che ricorre spesso nei laboratori avanzati. Nel **Tutorato 5 e 6**, quando si modellano sistemi come macchinette del caffè o controller di abbonamenti (SubscriptionController), la corretta implementazione dei Factory Method per validare e istanziare l'oggetto corretto (es. creare un **AbbonatoMensile** o **Annuale** in base all'input utente) diventa un requisito fondamentale di valutazione. Il pattern State, in particolare, è eccellente per modellare i *menu di gioco* o l'interfaccia utente, dove l'intero sistema si comporta in modo diverso a seconda che ci si trovi nello stato "MenuPrincipale", "InGioco" o "Pausa".

16 Lezione 16 (Lab 3): Esercitazione su eccezioni, generics e pattern

16.1 Spiegazione teorica e Struttura del Laboratorio

Questo terzo laboratorio rappresenta un punto di sintesi fondamentale: l'obiettivo è imparare a unire i concetti architetturali (ereditarietà e interfacce) con la gestione della sicurezza (eccezioni) e la generalizzazione del codice (generics).

Quando si progetta un sistema complesso e robusto (come richiesto in sede d'esame), il programmatore deve padroneggiare:

- **Robustezza (*Fail-fast*):** Ogni operazione che altera lo stato del sistema deve essere rigidamente protetta. Se un'operazione non è valida, il metodo non deve limitarsi a fallire in silenzio (restituendo `false` o `null`), ma deve **lanciare un'eccezione** chiara ed esplicativa.
- **Generalizzazione (Generics):** Se una classe deve gestire collezioni o operazioni su più tipi di oggetti, va resa parametrica (usando `<T>`). Questo permette di riutilizzare la stessa logica di gestione (es. un `Manager<T>`) per entità diverse, garantendo al contempo che il compilatore controlli i tipi corretti (*Type Safety*).
- **Gestione del flusso d'errore:** Imparare dove istanziare (`throw`) l'eccezione (solitamente nel *Model*, ovvero nelle classi base) e dove catturarla (`catch`) (solitamente nel *Controller* o nella *View*) per informare l'utente finale.

16.2 Implementazione pratica: Classe Generica Sicura

Di seguito un classico schema di laboratorio che unisce Generics ed Eccezioni per creare un contenitore con capienza massima, utile per inventari, party di giocatori o registri.

```
// 1. Definizione dell'eccezione di dominio (Checked)
public class CapacityExceededException extends Exception {
    public CapacityExceededException(String message) {
        super(message);
    }
}

// 2. Classe Parametrica (Generics)
public class SafeManager<T> {
    private final List<T> items;
    private final int maxCapacity;

    public SafeManager(int maxCapacity) {
        this.items = new ArrayList<>();
        this.maxCapacity = maxCapacity;
    }

    // Il metodo dichiara esplicitamente la possibilità di fallire
    public void add(T item) throws CapacityExceededException {
        if (items.size() >= maxCapacity) {
            // Regola di business violata: lancia eccezione Checked
            throw new CapacityExceededException("Capienza massima raggiunta!");
        }
        if (item == null) {
            // Errore del programmatore: lancia eccezione Unchecked
            throw new IllegalArgumentException("L'elemento non può essere null.");
        }
        items.add(item);
    }
}
```

16.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 6**, l'applicazione pratica di queste logiche si concretizza nel complesso sistema gestionale delle **Missioni (Quests)**.

16.3.1 Il Sistema QuestLog e le Eccezioni di Dominio

L'esercizio richiede di implementare un `QuestController` che supervisiona un `QuestLog` (una lista di missioni per il personaggio `Geraldo`). Il design del sistema prevede la definizione di una gerarchia di eccezioni molto precisa, derivata da una `QuestException` base:

- **AlreadyCompletedException**: lanciata se si tenta di accettare, svolgere o riscuotere una missione già contrassegnata come chiusa.
- **InsufficientLevelException**: lanciata se il livello di `Geraldo` è troppo basso per affrontare una `ContractQuest` o una `DLCQuest`.
- **PreviousQuestNotCompletedException**: fondamentale per le `MainQuest`, lanciata quando si tenta di avviare un capitolo della storia principale senza aver terminato quello propedeutico.

Analisi del design e validità per l'esame: Questo approccio è perfetto. Invece di usare infiniti `if` che restituiscono `false` nel metodo `acceptQuest()` (lasciando il programma all'oscuro del *perché* l'operazione sia fallita), il lancio di eccezioni specifiche permette al `Controller` di catturarle in un blocco `try-catch` e mostrare alla `ConsoleView` il motivo esatto del fallimento (es. "Devi prima completare la missione precedente!").

17 Lezione 17: Uguaglianza e Ordinamento

17.1 Spiegazione teorica

In Java, confrontare due oggetti o metterli in ordine richiede molta più attenzione rispetto all'utilizzo dei classici operatori matematici sui tipi primitivi. Dobbiamo distinguere due concetti di uguaglianza e comprendere i contratti previsti dalle interfacce di sistema.

- **Identità (Operatore ==)**: L'operatore `==` verifica se due riferimenti puntano **esattamente allo stesso indirizzo di memoria** (cioè se sono lo stesso identico oggetto nello *Heap*).
- **Uguaglianza Logica (Metodo equals())**: Verifica se due oggetti, pur vivendo in due locazioni di memoria differenti, hanno "lo stesso valore" secondo la logica del nostro dominio (es. due blocchi di terra con le stesse coordinate, o due giocatori con lo stesso username). Di default, la classe base `Object` implementa `equals()` usando semplicemente `==`. Per ottenere l'uguaglianza logica, dobbiamo fare l'**override** del metodo.

- **Il contratto di hashCode():** Se facciamo l'override di `equals()`, **dobbiamo obbligatoriamente** fare l'override anche di `hashCode()`. Questo metodo restituisce un numero intero usato per indicizzare l'oggetto in strutture dati ad alte prestazioni come `HashSet` o `HashMap`. La regola fondamentale di Java impone che: *"Se due oggetti sono logicamente uguali tramite `equals()`, allora i loro `hashCode()` devono restituire lo stesso numero"*. Se rompiamo questo patto, l'oggetto non verrà mai trovato all'interno delle collezioni.
- **Interfacce per l'Ordinamento:**
 - **Comparable<T>:** Si usa per definire l'**ordinamento naturale** della classe (es. ordine alfabetico per le Stringhe). La classe stessa implementa questa interfaccia e fornisce il metodo `compareTo(T o)`. Restituisce un numero negativo (se `this < o`), zero (se uguali), o positivo (se `this > o`).
 - **Comparator<T>:** È un oggetto esterno (spesso una classe anonima o lambda) che permette di definire **ordinamenti alternativi** al volo (es. ordinare i giocatori prima per punteggio, e a parità di punteggio per nome alfabeticamente), agendo in modo identico allo *Strategy Pattern*.

17.2 Implementazione del codice

17.2.1 Override sicuro di equals e hashCode

Vediamo come si implementa la logica per considerare uguali due oggetti `Point2D`.

```
import java.util.Objects;

public class Point2D {
    private final int x, y;
    public Point2D(int x, int y) {
        this.x = x; this.y = y;
    }
    @Override
    public boolean equals(Object obj) {
        // 1. Controllo di identità (ottimizzazione)
        if (this == obj) return true;
        // 2. Controllo di validità del tipo e null (evita ClassCastException)
        if (obj == null || this.getClass() != obj.getClass()) return false;

        // 3. Casting sicuro al tipo dinamico corretto
        Point2D other = (Point2D) obj;

        // 4. Confronto logico dello stato
        return this.x == other.x && this.y == other.y;
    }
    @Override
    public int hashCode() {
        // Usiamo il metodo utility di Java per generare un hash coerente
        return Objects.hash(x, y);
    }
}
```

17.2.2 Implementazione di Comparable (Ordinamento Naturale)

```
public class Player implements Comparable<Player> {
    private String username;
    private int score;

    public Player(String username, int score) {
        this.username = username;
        this.score = score;
    }

    @Override
    public int compareTo(Player other) {
        // Ordinamento decrescente: punteggio più alto per primo.
        // Se si usasse Integer.compare(this.score, other.score) sarebbe crescente.
        int scoreComparison = Integer.compare(other.score, this.score);

        // Se hanno lo stesso punteggio, ordino per ordine alfabetico naturale
        if (scoreComparison == 0) {
            return this.username.compareTo(other.username);
        }
        return scoreComparison;
    }
}
```

17.2.3 Utilizzo di Comparator (Ordinamento Alternativo)

```
import java.util.Collections;
import java.util.List;
import java.util.Comparator;

public class Leaderboard {
    public void sortAlphabetically(List<Player> players) {
        // Usiamo un Comparator al volo per ignorare i punteggi
        Collections.sort(players, new Comparator<Player>() {
            @Override
            public int compare(Player p1, Player p2) {
                return p1.getUsername().compareTo(p2.getUsername());
            }
        });
    }
}
```

Commento: In questo blocco stiamo iniettando una logica esterna in `Collections.sort()`, scavalcando il `compareTo` di default della classe `Player`. Questo è estremamente flessibile. Nelle lezioni successive vedremo come semplificare questa sintassi usando le Lambda Expressions.

17.3 Esercizi e approfondimenti dai laboratori

La corretta stesura del metodo `equals()` è l'errore più comune in sede d'esame. Negli esercizi di architettura complessa (come il sistema `QuestController` del **Tutorato 6**), quando il testo richiede: *"Il controller non deve accettare missioni duplicate nel QuestLog"*, questo controllo si esegue chiamando `list.contains(quest)`.

Attenzione: il metodo `contains()` delle liste Java basa il suo controllo **esclusivamente sul metodo `equals()`**. Se vi dimenticate di fare l'override di `equals()` nella classe `Quest`, Java userà il confronto di identità e inserirà due missioni identiche (ma create in locazioni di memoria diverse), violando il requisito del tema d'esame.

18 Lezione 18: Comportamenti reattivi e operazioni esterne (Observer e Visitor)

18.1 Spiegazione teorica

Man mano che i sistemi diventano complessi, si presentano due problemi ricorrenti: come far comunicare oggetti diversi senza accoppiarli (senza che debbano conoscersi intimamente) e come aggiungere nuove operazioni a una gerarchia di classi senza doverla modificare continuamente. La soluzione risiede in due Design Pattern comportamentali avanzati.

- **Observer Pattern (Comportamenti Reattivi):** Risolve il problema della notifica di stato. Definisce una dipendenza "uno-a-molti" tra oggetti in modo che, quando un oggetto (*Subject* o *Observable*) cambia il suo stato interno, tutti gli oggetti dipendenti (*Observers*) vengano notificati e aggiornati automaticamente.
 - Consente un **accoppiamento debole** (loose coupling): il *Subject* sa solo che i suoi osservatori implementano una certa interfaccia, non conosce le loro classi concrete.
 - È il cuore dei sistemi ad eventi (es. listener dei bottoni nelle interfacce grafiche) e dell'architettura MVC che vedremo in seguito.
- **Visitor Pattern (Operazioni Esterne):** Risolve il problema di dover aggiungere nuove operazioni su una struttura di oggetti eterogenei senza alterare le classi degli oggetti stessi.
 - Utilizza una tecnica chiamata **Double Dispatch**: l'operazione da eseguire dipende sia dal tipo del *Visitor* (l'operazione) sia dal tipo dell'*Element* (l'oggetto che riceve la visita).
 - Si usa quando la gerarchia degli elementi è molto stabile (es. la gerarchia dei blocchi o delle entità), ma le operazioni da eseguire su di essi cambiano o si espandono frequentemente (es. calcolare danni speciali, esportare dati in file, renderizzare graficamente).

18.2 Implementazione del codice

18.2.1 L'Observer Pattern (Player e AchievementSystem)

Vogliamo sbloccare un achievement quando il giocatore raccoglie i diamanti, ma non vogliamo che la classe `Player` contenga direttamente il codice degli achievement (violazione della singola responsabilità).

```
import java.util.ArrayList;
import java.util.List;

// 1. Interfaccia Observer (Chi ascolta)
public interface PlayerObserver {
    void onPlayerInventoryChanged(Player p);
}

// 2. Il Subject (Chi viene osservato)
public class Player {
    private List<PlayerObserver> observers = new ArrayList<>();
    private int diamonds = 0;

    public void addObserver(PlayerObserver o) {
        observers.add(o);
    }

    public void addDiamond() {
        this.diamonds++;
        notifyObservers(); // Notifica tutti quando lo stato cambia!
    }

    private void notifyObservers() {
        for (PlayerObserver o : observers) {
            o.onPlayerInventoryChanged(this); // Passa se stesso come contesto
        }
    }

    public int getDiamonds() { return this.diamonds; }
}

// 3. Un Observer concreto
public class AchievementSystem implements PlayerObserver {
    @Override
    public void onPlayerInventoryChanged(Player p) {
        if (p.getDiamonds() == 1) {
            System.out.println("ACHIEVEMENT UNLOCKED: DIAMONDS!");
        }
    }
}
```

Commento: Se in futuro vorremo aggiungere un sistema grafico che aggiorna l'icona dell'inventario, basterà creare una `GUIObserver` e aggiungerla al `Player`. Il codice del `Player` non dovrà subire alcuna modifica.

18.2.2 Il Visitor Pattern (Entity e DamageVisitor)

```
// 1. Interfaccia Visitor (L'operazione esterna)
public interface EntityVisitor {
    void visit(Zombie z);
    void visit(Creeper c);
}

// 2. Interfaccia Element (Chi accetta la visita)
public interface Entity {
    void accept(EntityVisitor visitor);
}

// 3. Elementi concreti (La gerarchia stabile)
public class Zombie implements Entity {
    @Override
    public void accept(EntityVisitor visitor) {
        visitor.visit(this); // Risoluzione del tipo esatto a runtime
    }
}

public class Creeper implements Entity {
    @Override
    public void accept(EntityVisitor visitor) {
        visitor.visit(this);
    }
}

// 4. Visitor concreto (La nuova operazione)
public class HolyDamageVisitor implements EntityVisitor {
    @Override
    public void visit(Zombie z) {
        System.out.println("Holy Damage vs Zombie: CRITICAL HIT! (x2 Damage)");
    }

    @Override
    public void visit(Creeper c) {
        System.out.println("Holy Damage vs Creeper: Normal Hit.");
    }
}
```

Commento: Senza il *Visitor*, per implementare il "danno sacro" avremmo dovuto usare bruttissimi `if` (e `instanceof` `Zombie`) o sporcare le classi originali aggiungendo `takeHolyDamage()`. Con il *Visitor*, ogni nuova logica complessa risiede nella propria classe separata.

18.3 Collegamento con i tutorati e sviluppi futuri

Anche se il **Tutorato 4 e 5** non chiedono esplicitamente di implementare un *Visitor*, introducono le fondamenta per l'architettura dei successivi laboratori. L'*Observer Pattern*,

in particolare, è il ponte concettuale che ci guiderà dritti alla **Lezione 22**: il pattern **Model-View-Controller (MVC)**. Nel MVC, la *View* (l'interfaccia grafica) si comporta come un *Observer* in attesa che il *Model* (i dati di gioco) la notifichi di eventuali cambiamenti, separando in modo perfetto la logica dalla grafica.

19 Lezione 19 (Lab 4): Esercitazione su Collections e Pattern

19.1 Spiegazione teorica e Struttura del Laboratorio

Il quarto laboratorio è un banco di prova per verificare la comprensione delle lezioni 17 e 18. Viene richiesto di gestire un sistema che non ha un solo tipo di dato e un solo comportamento, ma una gerarchia complessa in cui gli oggetti devono essere raggruppati, filtrati, ordinati e manipolati da agenti esterni.

Le fasi del laboratorio si concentrano su:

- **Gestione Avanzata delle Liste:** Non basta più fare una semplice `List<Entity>`. Si richiede di effettuare ricerche (es. trovare l'elemento con il costo maggiore), calcoli aggregati (es. la somma totale degli abbonamenti) e ordinamenti personalizzati tramite `Comparator`.
- **Separazione delle responsabilità (Pattern):** Quando una logica esterna deve agire su un oggetto (es. un Tecnico che ripara un Dispositivo, o uno Sconto che si applica a un Abbonamento), non bisogna "sporcare" la classe del modello dati inserendovi la logica dell'operatore. Si sfruttano interfacce e pattern (come lo `Strategy` o l'embrione del `Visitor`).

19.2 Implementazione pratica: Collections e Ordinamento

Prendendo ispirazione dall'esercizio della gestione degli `Abbonati`, vediamo come strutturare il modello e l'ordinamento.

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

// 1. Classe Base Astratta che implementa Comparable (Ordinamento Naturale)
public abstract class Abbonato implements Comparable<Abbonato> {
    protected String nome;
    protected double costoBase;

    public Abbonato(String nome, double costoBase) {
        this.nome = nome;
        this.costoBase = costoBase;
    }

    public abstract double getCostoFinale();
    public String getNome() { return this.nome; }

    // Ordinamento naturale di default: per nome (alfabetico)
    @Override
    public int compareTo(Abbonato other) {
        return this.nome.compareTo(other.nome);
    }
}

// 2. Sottoclasse specifica
public class Studente extends Abbonato {
    public Studente(String nome) { super(nome, 50.0); }

    @Override
    public double getCostoFinale() {
        return this.costoBase * 0.8; // 20% di sconto studenti
    }
}

```

Per ordinare dinamicamente gli abbonati non per nome, ma per *costo*, si crea un `Comparator` esterno:

```
public class GymManager {
    private List<Abbonato> iscritti = new ArrayList<>();

    public void aggiungi(Abbonato a) { iscritti.add(a); }

    public void stampaOrdinatiPerCosto() {
        // Uso di un Comparator con classe anonima (o Lambda in Java 8+)
        Collections.sort(iscritti, new Comparator<Abbonato>() {
            @Override
            public int compare(Abbonato a1, Abbonato a2) {
                return Double.compare(a1.getCostoFinale(), a2.getCostoFinale());
            }
        });

        for (Abbonato a : iscritti) {
            System.out.println(a.getNome() + " - " + a.getCostoFinale());
        }
    }
}
```

19.3 Esercizi e approfondimenti dal tutorato (Lab 4)

Il **Tutorato 4** esplora questi concetti attraverso due esercizi emblematici:

19.3.1 Esercizio 1: Palestra e Categorie

La gerarchia `Abbonato`, `Studente` e `Atleta` richiede di sfruttare il polimorfismo per il calcolo della retta. Lo `Studente` applica una riduzione percentuale, mentre l'`Atleta` potrebbe avere costi aggiuntivi in base alla `Categoria` sportiva. Raccogliendo tutti in una `List<Abbonato>`, il sistema stampa i bilanci aziendali iterando sulla collezione senza bisogno di operare casting, delegando a runtime il calcolo al tipo dinamico.

19.3.2 Esercizio 3: Dispositivi e Tecnici (Verso il Visitor)

L'esercizio 3 modella un `Tecnico` che ripara un `Dispositivo` (come uno `Smartphone`). Questa struttura è un trampolino di lancio per il Pattern **Visitor**: Invece di avere un metodo `ripara()` dentro lo `Smartphone` (che violerebbe la singola responsabilità, in quanto un telefono non si auto-ripara), la logica di riparazione appartiene al `Tecnico`. Il `Tecnico` riceve l'interfaccia `Dispositivo` e applica la sua strategia, mantenendo le classi di dominio (il telefono) puramente dedicate al contenimento dello stato.

20 Lezione 20: Programmazione Funzionale (Interfacce Funzionali e Lambda)

20.1 Spiegazione teorica

Fino a Java 7, l'unico modo per passare un "comportamento" come parametro a un metodo era creare un oggetto (spesso tramite una *Classe Anonima*) che implementasse una specifica interfaccia. Questo portava a scrivere molto codice ripetitivo (boilerplate). Con Java 8, è stato introdotto il paradigma funzionale per rendere il codice più conciso, leggibile e orientato al *cosa* fare piuttosto che al *come* farlo (approccio dichiarativo).

- **Interfacce Funzionali (SAM - Single Abstract Method):** Un'interfaccia si definisce "funzionale" se contiene **esattamente un solo metodo astratto** (può comunque contenere infiniti metodi `default` o `static`).
 - L'annotazione opzionale `@FunctionalInterface` dice al compilatore di verificare che la regola del singolo metodo sia rispettata.
 - Interfacce classiche come `Runnable` (ha solo `run()`) o `Comparator` (ha solo `compare()`) sono interfacce funzionali.
- **Espressioni Lambda (->):** Sono "funzioni anonime" che permettono di implementare al volo il singolo metodo di un'interfaccia funzionale, senza dover dichiarare una classe intera.
 - **Sintassi base:** `(parametri) -> { corpo della funzione }`
 - Se c'è un solo parametro, le parentesi tonde si possono omettere: `p -> { ... }`
 - Se il corpo ha una sola istruzione, le parentesi graffe e il `return` si possono omettere: `(a, b) -> a + b`
- **Interfacce Funzionali di Sistema (`java.util.function`):** Java fornisce interfacce standard pronte all'uso per non doverne creare di nuove ogni volta:
 - `Predicate<T>`: Prende in input un `T` e restituisce un `boolean` (utile per filtrare).
 - `Consumer<T>`: Prende in input un `T` e non restituisce nulla (utile per stampare o modificare).
 - `Function<T, R>`: Prende un `T` e restituisce un `R` (utile per mappare/trasformare).
- **Method References (::):** Se una lambda fa solo una cosa, ovvero richiamare un metodo già esistente, si può usare una sintassi ancora più compatta: `Classe::metodo` (es. `System.out::println`).

20.2 Implementazione del codice

20.2.1 Refactoring: Dalle Classi Anonime alle Lambda

Riprendiamo l'esempio del `Comparator` della Lezione 19 e vediamo come si evolve.

```
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

public class LambdaRefactoring {
    public void sortPlayers(List<Player> players) {
        // APPROCCIO VECCHIO (Java 7): Classe Anonima
        Collections.sort(players, new Comparator<Player>() {
            @Override
            public int compare(Player p1, Player p2) {
                return Integer.compare(p1.getScore(), p2.getScore());
            }
        });

        // NUOVO APPROCCIO (Java 8+): Espressione Lambda
        // Il compilatore "capisce" che p1 e p2 sono di tipo Player
        Collections.sort(players, (p1, p2) -> Integer.compare(p1.getScore(),
                                                                p2.getScore()));

        // APPROCCIO MODERNO CON METHOD REFERENCE (Ottimale)
        players.sort(Comparator.comparingInt(Player::getScore));
    }
}
```

Commento: Da 7 righe di codice a 1 sola riga. L'uso delle lambda rende l'intento del programmatore ("ordina per punteggio") immediatamente evidente, nascondendo la complessità strutturale.

20.2.2 Uso pratico di Predicate e Consumer

Vediamo come passare un blocco di codice come parametro per filtrare un inventario.

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;
import java.util.function.Consumer;

public class Inventory {
    private List<Item> items = new ArrayList<>();

    // Questo metodo accetta una CONDIZIONE e un'AZIONE
    public void processItems(Predicate<Item> condition,
                           Consumer<Item> action) {
        for (Item item : items) {
            // Se l'oggetto rispetta la condizione...
            if (condition.test(item)) {
                // ...esegui l'azione
                action.accept(item);
            }
        }
    }

    public static void testLambda() {
        Inventory inv = new Inventory();

        // Uso le lambda per definire la logica "al volo"
        // Condizione: l'arma è rotta (durability == 0)
        // Azione: stampala a schermo
        inv.processItems(
            item -> item.getDurability() == 0,
            item -> System.out.println("Item rotto: " + item.getName())
        );
    }
}
```

20.3 Collegamento con i laboratori futuri (Verso le GUI)

Sebbene il **Tutorato 5 e 6** utilizzino ampiamente le Collections per la gestione dei dati (es. filtrare le **Quest** attive o concluse), la vera potenza delle espressioni Lambda esploderà nelle lezioni finali del corso, dedicate all'interfaccia grafica (JavaFX) e all'architettura MVC.

Quando un utente preme un pulsante nell'interfaccia (es. `nameButton`), il sistema deve scatenare un evento. Il modo moderno per farlo è passare un'espressione lambda al bottone: `nameButton.addEventHandler(MouseEvent.MOUSE_CLICKED, event -> mc.logic());`. Senza le lambda, scrivere l'interfaccia grafica richiederebbe la creazione di decine di classi ridondanti solo per intercettare i click del mouse.

21 Lezione 21: Architettura del Software (Pattern Model-View-Controller)

21.1 Spiegazione teorica

L'apice del "programming in the large" orientato agli oggetti si raggiunge con la creazione di interfacce grafiche (GUI). Quando si progetta un'applicazione interattiva, il rischio maggiore è creare "classi dio" (God Classes) che contengono contemporaneamente la logica del programma e il codice per disegnare i bottoni sullo schermo. Questo rende il codice impossibile da testare, mantenere o riutilizzare.

Per risolvere questo problema, l'industria software adotta il pattern architetturale **Model-View-Controller (MVC)**, che impone una rigida separazione delle responsabilità (Separation of Concerns):

- **Model (Il Modello):** Contiene i dati puri, lo stato dell'applicazione e le regole di business (es. la classe `Player`, l'inventario, la vita). **Non sa assolutamente nulla dell'interfaccia grafica.** Non importa `java.awt` o `javafx`. Se i suoi dati cambiano, notifica gli interessati usando l'*Observer Pattern*.
- **View (La Vista):** È la rappresentazione visiva (i bottoni, i testi, le finestre). Il suo unico compito è mostrare i dati del Modello all'utente. Si comporta da *Observer*: quando il Modello la avvisa di un cambiamento, la Vista si ridisegna. **Non contiene alcuna logica di business** (non decide se un giocatore muore, si limita a disegnare la barra della vita vuota).
- **Controller (Il Controllore):** È il "direttore d'orchestra" e gestisce l'input dell'utente (click del mouse, pressione dei tasti). Quando l'utente preme un bottone sulla Vista, l'evento viene catturato dal Controller, che a sua volta invoca i metodi appropriati per modificare il Modello.

21.2 Implementazione del codice

A lezione è stato illustrato l'esempio `multiInputsMVC`, che mostra come la Vista deleghi le azioni tramite eventi e lambda expressions.

21.2.1 La Vista (Intercettare l'input visivo)

```
package lecture22.multiInputsMVC.view;

import javafx.scene.control.Button;
import javafx.scene.input.MouseEvent;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;

public class MultiStoneBlockView {
    private Button nameButton = new Button("Mine Block");

    // Il costruttore o un metodo di inizializzazione riceve il Controller
    public void setupEvents(MultiStoneBlockController mc) {

        // Uso delle Lambda per collegare l'evento UI al Controller
        this.nameButton.addEventHandler(MouseEvent.MOUSE_CLICKED, event -> {
            mc.logic(); // La Vista non sa COSA succede,
                        // avvisa solo il Controller
        });

        this.addEventHandler(KeyEvent.KEY_PRESSED, event -> {
            if (event.getCode() == KeyCode.N) {
                mc.logic();
            }
        });
    }
}
```

21.2.2 Il Controller (La logica di smistamento)

```
package lecture22.multiInputsMVC.controller;

public class MultiStoneBlockController {
    private Model model;
    private MultiStoneBlockView view;

    public MultiStoneBlockController(Model model, MultiStoneBlockView view) {
        this.model = model;
        this.view = view;
        this.view.setupEvents(this); // Collega gli eventi
    }

    // Metodo invocato dalla Vista
    public void logic() {
        try {
            // Il Controller altera il Modello
            model.mine();
        } catch (BlockAlreadyBrokenException e) {
            // Il Controller gestisce gli errori
            // e dice alla Vista di mostrare un popup
            view.showError("Il blocco è già rotto!");
        }
    }
}
```

Commento architetturale: In questo snippet riassuntivo vediamo come tutti gli argomenti del corso convergano. Le eccezioni garantiscono la robustezza del Modello, le Lambda gestiscono gli eventi nella Vista, e il Controller fa da ponte sicuro, applicando il polimorfismo.

21.3 Esercizi e approfondimenti dal tutorato

Nei tutorati finali (**Tutorato 5 e Tutorato 6**), la struttura delle cartelle impone nativamente il pattern MVC. Ad esempio, nell'esercizio delle Quest del Tutorato 6:

- La cartella `model` contiene le enum (`Luoghi`, `Mostri`), le eccezioni di dominio (`QuestException`) e la logica delle Quest.
- La cartella `view` contiene una `ConsoleView.java`. Anche senza un'interfaccia grafica vera e propria, la separazione viene rispettata: la `ConsoleView` stampa a schermo il testo (es. "Scegli un'azione") e legge lo `Scanner`.
- La cartella `controller` contiene il `QuestController.java`, che riceve i comandi testuali dalla `ConsoleView` e invoca i metodi su `Geraldo` o sul `QuestLog`.

All'esame, violare questa separazione (ad esempio inserendo un `System.out.println` o uno `Scanner` all'interno delle classi del `model`) comporta gravi penalità di valutazione, poiché distrugge la riusabilità del codice.

22 Lezione 22 (Lab 5): Esercitazione su Layout grafici e implementazione MVC

22.1 Spiegazione teorica e Struttura del Laboratorio

Il quinto laboratorio è incentrato sull'applicazione rigorosa del pattern Model-View-Controller (MVC). A differenza della pura teoria vista in precedenza, qui l'obiettivo è comprendere come far scalare l'architettura passando da esempi didattici a veri e propri progetti organizzati su più file e package.

Il laboratorio è strutturato in tre fasi di analisi e sviluppo:

- **Fase 1: Che layout usare:** Si discute su come organizzare visivamente gli elementi. Nelle interfacce grafiche moderne (come JavaFX), la *View* non è un monolito, ma è composta da vari contenitori (HBox, VBox, ecc.) per disporre pulsanti e testi.
- **Fase 2: Come fare MVC semplice:** Creazione dello scheletro base in cui un singolo *Controller* gestisce le interazioni tra un *Model* semplice e una singola *View*.
- **Fase 3: Come fare MVC complesso:** Estensione del sistema per gestire modelli ricchi (gerarchie di classi, composizione) ed eventi ramificati.

22.2 Esercizi e approfondimenti dal tutorato (Lab 5)

Il **Tutorato 5** cala questi concetti nella pratica fornendo un esempio eccellente di "MVC Complesso" per la gestione di un sistema di abbonamenti a ventilatori.

22.2.1 Architettura dei Package (Il "Segreto" per l'Esame)

La corretta suddivisione del progetto in directory (package) è il primo fondamentale criterio di valutazione all'esame. Vediamo l'albero dei file del progetto 5TutP2 e analizziamone la logica:

```

src/
model/                                // IL MODELLO (Logica e Dati Puri)
    alimentazioni/                    // -> Sottosistema compositivo
        AbstractAlimentazione.java
        AlimentazioneBatteria.java
        AlimentazionePresa.java
    enums/                            // -> Costanti di dominio
        Marche.java
    fans/                             // -> Gerarchia degli oggetti principali
        AbstractFan.java
        PiantanaFan.java
        SoffittoFan.java
    Utente.java                       // -> Stato dell'attore principale
    Vetrina.java                      // -> Il contenitore centrale (Database/Manager)

view/                                // LA VISTA (Interfaccia Utente)
    ConsoleView.java                 // -> Disegna l'interfaccia (anche se testuale)

controller/                          // IL CONTROLLER (Smistamento e Validazione)
    SubscriptionController.java       // -> Coordina Vetrina e ConsoleView

Main.java                            // ENTRY POINT (Inizializzazione)

```

Analisi del Flusso MVC Complesso nel Tutorato:

1. **Inizializzazione (Main.java):** All'avvio, il programma non deve contenere logica. Si limita a istanziare la *Vetrina* (Model) e la *ConsoleView* (View), passandole come argomenti al costruttore del *SubscriptionController*.
2. **Visualizzazione (ConsoleView):** La vista non legge direttamente gli *AbstractFan* per stamparli. L'accoppiamento deve essere debole. La vista delega al controller la richiesta dei dati da mostrare.
3. **Interazione (SubscriptionController):** Quando un *Utente* decide di sottoscrivere un abbonamento, la *View* cattura l'input (es. tramite uno *Scanner* o un bottone) e invoca un metodo sul *Controller*. Il *Controller* invoca i metodi del *Model* (es. la classe *Vetrina*).
4. **Gestione Eccezioni:** Se la *Vetrina* lancia un'eccezione (es. "Modello esaurito" o "Credito insufficiente"), il *Controller* la cattura nel blocco *try-catch* e comanda alla *View* di renderizzare un messaggio d'errore all'utente.

Questo schema garantisce che, se domani volessimo sostituire la *ConsoleView* con un'interfaccia JavaFX, l'intera cartella *model* e gran parte del *controller* non richiederebbero alcuna modifica, realizzando la promessa dell'OOP.

23 Lezione 23 (Lab 6): Conclusione ed Esercitazione Finale

23.1 Spiegazione teorica e Struttura del Laboratorio

L'ultimo laboratorio del corso non introduce nuovi costrutti sintattici, ma rappresenta la prova generale prima dell'esame. L'obiettivo è dimostrare di saper orchestrare tutti i concetti visti finora (*Programming in the Large*) all'interno di un'unica architettura robusta e manutenibile.

Un tema d'esame tipico (o un progetto finale) richiede di applicare simultaneamente:

- **Architettura MVC:** Separazione rigorosa tra chi detiene i dati (**Model**), chi li mostra (**View**) e chi gestisce le interazioni (**Controller**).
- **Ereditarietà e Polimorfismo:** Costruzione di gerarchie profonde (es. `AbstractQuest` → `MainQuest`, `DLCQuest`) sfruttando il *Dynamic Binding* per evitare cast e controlli di tipo a runtime.
- **Design Patterns:** Uso del *Template Method* per definire scheletri algoritmici, dello *Strategy* o del *Visitor* per operazioni esterne, e di *Factory* per la creazione degli oggetti.
- **Robustezza (Eccezioni e Invarianti):** Uso di eccezioni custom (`Checked` e `Unchecked`) per bloccare transizioni di stato non valide e garantire che gli oggetti non assumano mai stati illogici.
- **Collezioni e Generics:** Uso avanzato di Liste per gestire inventari, log di missioni o database, abbinati a `Comparable` e `Comparator` per l'ordinamento.

23.2 Caso Studio Riassuntivo: Il Sistema QuestLog (Tutorato 6)

Il **Tutorato 6** (basato sulla gestione delle missioni di **Geraldo**) è il prototipo perfetto di un esame da 30 e lode. Analizziamo come i vari pezzi del puzzle si incastrano in un'architettura completa.

23.2.1 1. Il Modello: Ereditarietà e Invarianti

Si parte definendo le interfacce e le classi astratte. Una `AbstractQuest` definisce i campi base (titolo, ricompensa) e delega alle sottoclassi i requisiti specifici.

```
package model.quests;
import model.exceptions.*;

public abstract class AbstractQuest implements Quest {
    private final String title;
    private boolean isCompleted = false;

    public AbstractQuest(String title) {
        this.title = title;
    }

    // Il metodo lancia eccezioni custom se le regole di business sono violate
    public void completeQuest(int playerLevel) throws QuestException {
        if (isCompleted) {
            throw new AlreadyCompletedException("Missione già completata!");
        }
        if (!checkRequirements(playerLevel)) {
            throw new InsufficientLevelException("Livello troppo basso.");
        }
        this.isCompleted = true;
    }

    // Hook polimorfico (Template Method) che le sottoclassi devono implementare
    protected abstract boolean checkRequirements(int playerLevel);
}
```

23.2.2 2. Il Modello: Gestione delle Collezioni

La classe `QuestLog` fa da "Database" (Spawner/Manager) incapsulando una `List<Quest>` e nascondendo i dettagli implementativi all'esterno.

```
package model;
import model.quests.Quest;
import java.util.ArrayList;
import java.util.List;

public class QuestLog {
    private List<Quest> activeQuests = new ArrayList<>();

    public void addQuest(Quest q) {
        // L'operatore equals() deve essere stato sovrascritto in Quest!
        if (!activeQuests.contains(q)) {
            activeQuests.add(q);
        }
    }
}
```

23.2.3 3. Il Controller: Il Direttore d'Orchestra

Il `QuestController` intercetta le richieste dell'utente (provenienti dalla *View*), invoca i metodi del *Model* e gestisce eventuali problemi in modo "pulito", catturando le eccezioni.

```
package controller;
import model.Geraldo;
import model.exceptions.QuestException;
import view.ConsoleView;

public class QuestController {
    private Geraldo player;
    private ConsoleView view;

    public QuestController(Geraldo player, ConsoleView view) {
        this.player = player;
        this.view = view;
    }

    public void attemptQuestCompletion(int questIndex) {
        try {
            // Delega la logica al modello
            player.getLog().completeQuestAt(questIndex, player.getLevel());
            view.showMessage("Successo! Hai completato la missione.");
        } catch (QuestException e) {
            // Cattura l'errore di dominio e aggiorna la UI
            view.showError("Fallimento: " + e.getMessage());
        }
    }
}
```

23.3 Conclusione e Consigli per l'Esame

1. **Leggere prima i Sostantivi e i Verbi:** Durante la lettura del testo d'esame, cerciate i sostantivi (diventeranno `Classi` o `Enum`) e i verbi (diventeranno `Metodi`).
2. **Disegnare l'albero delle directory:** Prima di scrivere una singola riga di codice, create i package `model`, `view` e `controller`.
3. **Incapsulamento totale:** Tutti i campi devono essere `private`. Usate `final` ovunque sia possibile. Fornite i `getter` solo se strettamente necessari per la View.
4. **Nessun `if` sui tipi:** Se vi trovate a scrivere `if (obj instanceof T)` oppure a fare `(T) obj`, fermatevi. State sbagliando l'architettura. Spostate quel comportamento in un metodo polimorfico o usate un `Visitor`.